A STRUCTURED 6-MONTH ROADMAP

COMPETITIVE PROGRAMMING · CODEFORCES

Codeforces Rating *Boosting* Guide

From contest rank **4000–5000** to **Expert (1600)** in six months of focused practice—10–15 hours per week, with deep dives on **DP** and **graph algorithms**.

6	24	34	80+
MONTHS	WEEKS	CHAPTERS	PROBLEMS

DYNAMIC PROGRAMMING · GRAPH ALGORITHMS · BINARY SEARCH · GREEDY · CONTEST STRATEGY

PREPARED FOR

Pupil & Specialist Climbers

TARGET

Expert — 1600

Table of Contents

Preface — How to Use This Guide 10

Who This Guide Is For . . . 10

How To Read This Guide . . . 10

Notation Conventions . . . 11

The Six-Month Commitment . . . 11

Chapter 1 — Understanding the Codeforces Rating System 13

Rating Bands and Titles . . . 13

What Contest Position 4000-5000 Means . . . 13

Why Rating Swings Happen . . . 14

The Role of Contest Frequency . . . 14

Chapter 2 — Self-Assessment and Goal Setting 15

The Diagnostic . . . 15

The Rating Ladder . . . 15

Tracking Progress . . . 16

Setting SMART Goals . . . 16

Chapter 3 — Mindset and Psychology of Rating Climbing 16

Tilt Management . . . 16

Rating Anxiety . . . 17

Focusing on Skills, Not Numbers . . . 17

Plateau Periods . . . 17

Building Resilience . . . 18

Chapter 4 — Setting Up Your Practice Environment 18

C++ Compiler and Flags . . . 18
Template File Structure . . . 19
Debug Macro . . . 19
Stress Testing . . . 20
Editor and Workflow . . . 20
Codeforces Interface Tips . . . 21

Chapter 5 — Time Complexity and Big-O Analysis 23

The 10^8 Rule of Thumb . . . 23
Counting Operations Accurately . . . 24
Common Complexity Traps . . . 24
Amortised Analysis . . . 24

Chapter 6 — C++ STL for Competitive Programming 25

Containers . . . 25
Algorithms . . . 26
When to Use Which Container . . . 26
Common STL Bugs . . . 26
Built-in Functions . . . 27

Chapter 7 — Implementation and Bug-Free Coding 27

Translating Ideas to Code . . . 27
Defensive Coding . . . 28
Common Bug Patterns . . . 28
Debugging Techniques . . . 29
Writing Short Clean Code . . . 29

Chapter 8 — Greedy Algorithms 29

When Greedy Works . . . 30

Classic Greedy Patterns . . . 30
Common Pitfalls . . . 30
Practice Problems by Pattern . . . 31

Chapter 9 — Binary Search and Ternary Search 31

Binary Search on a Sorted Array . . . 31
Binary Search on the Answer . . . 32
Ternary Search . . . 33
Monotonicity Recognition . . . 33

Chapter 10 — Two Pointers and Sliding Window 33

Two Pointers on Sorted Arrays . . . 33
Two Pointers on Sequences . . . 34
Sliding Window: Fixed and Variable . . . 34
Monotonic Deque . . . 35
When Two Pointers Replaces Binary Search . . . 35

Chapter 11 — DP Fundamentals 37

What DP Actually Is . . . 37
The Four-Step Method . . . 37
Top-Down vs Bottom-Up . . . 38
Common State Dimensions . . . 38

Chapter 12 — Classic DP Patterns 39

Pattern 1: Fibonacci-Style 1D DP . . . 39
Pattern 2: 0/1 Knapsack . . . 39
Pattern 3: Longest Increasing Subsequence . . . 40
Pattern 4: Longest Common Subsequence . . . 40
Pattern 5: Edit Distance . . . 40

Pattern 6: Coin Change 41

Chapter 13 — DP on Trees 41

The Basic Pattern 41

The Rerooting Technique 42

Common Tree DP Problems 42

Chapter 14 — Bitmask DP 42

Representing Subsets as Integers 43

The Subset Iteration Trick 43

Classic Bitmask DP Problems 43

When Bitmask DP Applies 44

Chapter 15 — Digit DP 44

The State 44

The Standard Template 45

Common Variants 45

Chapter 16 — Common DP Pitfalls and Optimisation 46

State Explosion 46

Wrong Base Cases 46

Missing Transitions 47

Dimension Reduction 47

Prefix-Sum Optimisation 47

Monotonic Queue Optimisation 47

When to Switch from DP to Greedy 47

Chapter 17 — Graph Representation and Traversal 50

Graph Representation 50

BFS — Breadth-First Search 50

DFS — Depth-First Search . . . 51
When BFS vs DFS . . . 51
Common Traversal Bugs . . . 52

Chapter 18 — Shortest Path Algorithms 52

Dijkstra's Algorithm . . . 52
Bellman-Ford Algorithm . . . 53
Floyd-Warshall Algorithm . . . 53
0-1 BFS . . . 54

Chapter 19 — Minimum Spanning Tree and Disjoint Set Union 54

Disjoint Set Union (DSU) . . . 54
Kruskal's Algorithm . . . 55
Prim's Algorithm . . . 55
DSU Applications Beyond MST . . . 56

Chapter 20 — Topological Sort and DAGs 56

Kahn's Algorithm (BFS-based) . . . 56
DFS-based Topological Sort . . . 57
DP on DAG . . . 57
Cycle Detection in Directed Graphs . . . 57

Chapter 21 — Strongly Connected Components and 2-SAT 57

Tarjan's SCC Algorithm . . . 58
Kosaraju's Algorithm . . . 58
Condensation DAG . . . 58
2-SAT . . . 58

Chapter 22 — Network Flow Basics 59

Max-Flow Min-Cut Theorem . . . 59
Ford-Fulkerson and Edmonds-Karp . . . 59
Bipartite Matching . . . 59
When Flow Appears in Problems . . . 60

Chapter 23 — Modular Arithmetic and Number Theory 62

The Mod Everywhere Discipline . . . 62
Modular Inverse . . . 62
Sieve of Eratosthenes . . . 63
GCD and LCM . . . 63
Prime Factorisation . . . 63

Chapter 24 — Combinatorics Basics 63

Binomial Coefficients Modulo p . . . 64
Pascal's Identity and Recurrence . . . 64
Stars and Bars . . . 64
Inclusion-Exclusion . . . 65

Chapter 25 — Month 1: Foundation and Speed (Weeks 1-4) 67

Week 1: Greedy and Sorting . . . 67
Week 2: Binary Search and Two Pointers . . . 67
Week 3: Math and Number Theory . . . 67
Week 4: STL and Implementation . . . 67

Chapter 26 — Month 2: DP Mastery (Weeks 5-8) 68

Week 5: DP Fundamentals and 1D DP . . . 68
Week 6: Knapsack and LIS/LCS . . . 68
Week 7: DP on Trees and Bitmask DP . . . 68
Week 8: Digit DP and DP Optimisation . . . 68

Chapter 27 — Month 3: Graph Mastery (Weeks 9-12) 69

Week 9: BFS/DFS and Applications . . . 69

Week 10: Shortest Paths . . . 69

Week 11: MST, DSU, and Topological Sort . . . 69

Week 12: SCC and Flow Basics . . . 69

Chapter 28 — Month 4: Mixed Topics and Speed (Weeks 13-16) 70

Week 13: Strings (Hashing, KMP Basics) . . . 70

Week 14: Data Structures (Segment Tree, BIT) . . . 70

Week 15: Constructive and Ad-hoc . . . 70

Week 16: Mixed Bag and Upsolving . . . 70

Chapter 29 — Month 5: Speed and Accuracy Drills (Weeks 17-20) 71

Week 17: Speed Drills . . . 71

Week 18: Hard Problems 1600-1800 . . . 71

Week 19: Implementation Marathon . . . 71

Week 20: Mock Contests . . . 71

Chapter 30 — Month 6: Contest Simulation and Refinement (Weeks 21-24) 72

Weeks 21-22: Full Virtual Contests . . . 72

Week 23: Targeted Weakness Review . . . 72

Week 24: Final Mock and Mental Prep . . . 72

Chapter 31 — Pre-Contest Preparation 74

The 24 Hours Before . . . 74

The 1 Hour Before . . . 74

Chapter 32 — In-Contest Time Management 74

Minutes 0-10: Read All Problems . . . 74

Minutes 10-60: Nail A and B Fast . . . 75
Minutes 60-90: C Attempt . . . 75
Minutes 90-120: D, C Optimisation, or Hacks . . . 75
Submission Timing Strategy . . . 75

Chapter 33 — Hacks and Counter-Hacks **75**

Hack Types . . . 76
Finding Hackable Solutions . . . 76
Counter-Hacks: Writing Robust Code . . . 76
The 12-2 Lockout Period . . . 76

Chapter 34 — Post-Contest Analysis and Upsolving **76**

The Post-Contest Routine . . . 77
Upsolving Rules . . . 77
The Mistakes Journal . . . 77

Appendix A — Topic-to-Rating Mapping **79**

Appendix B — Curated Problem List **79**

Greedy (Rating 1300-1600) . . . 80
Binary Search and Two Pointers (Rating 1300-1600) . . . 80
DP (Rating 1400-1700) . . . 81
Graphs (Rating 1400-1700) . . . 81
Math and Number Theory (Rating 1300-1600) . . . 82

Appendix C — Code Templates and Macros **82**

Master Template . . . 82
DSU Template . . . 83
Segment Tree (Point Update, Range Query) . . . 83
Fenwick Tree (BIT) . . . 84

Dijkstra 84

Appendix D — Recommended Resources and Final Words 84

Books 84

Codeforces Blogs 85

Other Judges 85

YouTube Channels 85

Final Words 85

Preface — How to Use This Guide

A 6-MONTH ROADMAP FROM CONTEST RANK 4000-5000 TO EXPERT

If you are reading this, you have probably been competing on Codeforces for a while, you place roughly between 4000th and 5000th in regular contests, and your rating sits somewhere in the Pupil-to-Specialist band (about 1200 to 1500). You are no longer a beginner, but every contest feels like a struggle to break past the same ceiling. This guide is built for exactly that situation. The goal is precise: take you from your current level to Expert (1600 rating) within six months of focused, deliberate practice.

The structure is intentional. We start with foundations — understanding the rating system, calibrating your current skill set, and setting up a practice environment that does not waste your time. Then we move through the core skills that every Specialist-to-Expert climber needs: time complexity analysis, the C++ standard library, greedy thinking, binary search, and two-pointer techniques. After that we deep-dive into the two topics that cause the most rating stagnation at your level — dynamic programming and graph algorithms — because they appear in nearly every Division 2 round and they reward systematic study far more than ad-hoc problem solving does.

The middle of the guide is a week-by-week, six-month practice plan organised around weekly themes. Each month has a clear focus, each week has a topic, and the difficulty ramps up steadily so you are always solving problems slightly above your current comfort zone. The plan assumes 10 to 15 hours of practice per week — enough to make real progress without burning out, and compatible with school or work obligations. The final parts of the guide cover contest-day strategy, post-contest analysis, and a reference appendix with topic-to-rating mappings, curated problem lists, and code templates you can paste into your editor today.

Who This Guide Is For

This guide assumes you can already solve Division 2 problems A and B consistently, occasionally solve C, and rarely solve D. You are comfortable with basic C++ syntax, know what a vector and a map are, and have at least a vague idea of what time complexity means. You do not need to be fluent in dynamic programming or graph theory — those are covered from first principles — but you should be past the stage where you struggle with loops, arrays, and basic input handling. If you are still at the Newcomer stage (under 1000 rating), work through a beginner resource like the USACO Guide or Learn Competitive Programming With C++ first, then come back here.

How To Read This Guide

The parts are designed to be read in order the first time, then revisited selectively. Read Part I in one sitting — it sets the framework. Treat Parts II through V as a reference textbook: when you are working on DP problems that week, re-read the relevant DP chapter; when you switch to graphs, re-read the graph chapters. Part VI is the operational plan — keep it open in a second window during your practice sessions. Part VII on contest strategy is worth re-reading the night before every rated round. The appendices are reference material you will return to repeatedly over the six months.

Throughout the guide you will find three types of emphasised content. Callout blocks in italic blue mark key insights that distil a chapter into a single sentence — if you only remember one thing from a chapter, it is the callout. Code blocks contain ready-to-paste C++ snippets, all of which have been tested against the GNU

C++20 compiler used by Codeforces. Tables summarise patterns, ratings, and problem lists in a form you can scan quickly during practice. Do not skip the tables — they encode most of the operational knowledge in the guide.

Notation Conventions

When we talk about problem ratings, we always use the Codeforces rating scale: a "1400 problem" means a problem rated 1400 on Codeforces, which typically corresponds to a Division 2 C or a Division 3 E/F. When we say "solve at your rating + 200", we mean pick problems whose Codeforces rating is approximately 200 points higher than your current displayed rating. The reason for the +200 offset is well-documented in the competitive programming community: solving problems just above your current level is the most efficient way to expand your skill ceiling without crossing into frustration territory.

When we refer to "contest position 4000-5000", we mean your finishing rank in a typical Codeforces Division 2 round with around 25,000 to 30,000 participants. In that contest size, finishing 4000th-5000th usually corresponds to solving problems A and B with a slow C attempt, landing you at a rating of roughly 1250-1450 depending on the round. The exact mapping varies because Codeforces uses an Elo-based system with contest-specific volatility, but the band is stable enough for planning purposes.

The Six-Month Commitment

Six months is a realistic timeframe for the climb from 1300 to 1600 — fast enough to stay motivated, slow enough to actually learn the material. It assumes you compete in roughly two rated rounds per week (about 50 contests over six months), solve 8 to 12 practice problems per week on top of contests, and up-solve at least one problem from every contest you do. The single biggest predictor of failure is not lack of talent — it is skipping the upsolving. If you do nothing else from this guide, do the upsolving. Every contest you walk away from without solving the next-hardest problem is a contest where your rating gain is capped at roughly the expected delta, which means you only climb when you have an unusually good day. Upsolving converts bad contests into learning and good contests into compounding gains.

> *Reading this guide without practising will not raise your rating. The guide is the map; the practice is the walking. Both are required.*

PART I

Foundations

Before you can climb, you need to understand the mountain. Part I covers the rating system, an honest self-assessment, the mindset that separates climbers from stagnators, and a practice environment that does not waste your time.

Chapter 1 — Understanding the Codeforces Rating System

WHY YOUR RATING MOVES THE WAY IT DOES

Codeforces uses a variant of the Elo rating system originally developed for chess. Every participant enters a contest with a current rating, and the system predicts a performance based on that rating. After the contest, your rating changes according to how your actual performance compared to the prediction. Beat the prediction and your rating rises; miss it and your rating falls. The size of the change depends on the magnitude of the surprise and on your rating volatility — a parameter that tracks how consistent your recent results have been. Newer accounts and inconsistent performers have higher volatility, which means bigger swings in both directions.

The most important practical consequence of this system is that your rating is not a direct measure of how many problems you solved — it is a measure of how many problems you solved relative to what the system expected you to solve based on your rating and the ratings of everyone else in the contest. This is why solving A, B, and C in a round where most people of your rating only solved A and B can produce a +60 delta, while solving the same A, B, and C in a round where most people of your rating also solved C might produce only +10 or even a small negative. The rating you gain depends on the difficulty of the problems you solved relative to your level, not on the raw count.

Rating Bands and Titles

Codeforces assigns titles based on rating, and these titles matter more than they should psychologically. The bands are: Newcomer (below 1200), Pupil (1200-1399), Specialist (1400-1599), Expert (1600-1899), Candidate Master (1900-2099), Master (2100-2299), International Master (2300-2399), Grandmaster (2400-2599), International Grandmaster (2600-2999), and Legendary Grandmaster (3000+). The colour of your handle on the Codeforces website changes with each band, and the community treats these colours as a rough proxy for skill. Your goal is to cross from Specialist grey-green to Expert blue.

The jump from high Specialist to Expert is one of the most common plateaus on Codeforces. It is also one of the most well-studied. The reason it is hard is that the problems at the 1500-1600 difficulty range start requiring more than direct application of a known technique. Below 1500, most problems are recognisable: you see the statement, you recognise the pattern, you implement it. Above 1500, problems increasingly require combining two or three ideas, or applying a known technique with a non-obvious twist. The skill that breaks you through is not learning more techniques — it is learning to recognise when to apply which technique, and how to combine them. That is what the rest of this guide teaches.

What Contest Position 4000-5000 Means

In a typical Codeforces Division 2 round with around 28,000 participants, finishing in position 4000 to 5000 corresponds roughly to a rating of 1250 to 1450, depending on the specific round. To translate this into problem-solving terms: you are reliably solving problems A and B (the two easiest), occasionally solving C, and rarely making meaningful progress on D. Your A and B solves are not always fast — sometimes you lose ten or fifteen minutes on a tricky B that should have taken five. Your C attempts, when they succeed, often

take the full contest time, leaving nothing for D.

The path out of this band is not mysterious. First, you need to solve A and B faster — within the first 25 minutes combined, not the first 50. That alone is worth roughly 50-100 rating points because the scoring formula rewards early submissions. Second, you need to solve C more than half the time, which means closing the gap on the techniques that appear at 1400-1600 difficulty: greedy with a non-obvious ordering, binary search on the answer, basic dynamic programming, simple graph traversal. Third, you need to attempt D in the rounds where C is solved early — even an unsuccessful attempt that you up-solve afterward builds the skill that eventually turns into D solves in contests.

Why Rating Swings Happen

If you have ever lost 80 rating in one contest and gained 70 in the next without feeling like you solved anything differently, you have experienced rating volatility. Volatility is highest for new accounts and for accounts with inconsistent recent results. It decays as you compete in more contests and as your results become more consistent. Practically, this means your first 20-30 contests will have larger swings than your later ones, and a single bad contest will hurt your rating more early in your career than later. The implication for climbers: do not over-index on any single contest result. Track your rolling average over the last 5 contests, not the delta of the most recent one.

A second source of swings is the difficulty distribution of the round itself. Some rounds have an unusually hard C, which means everyone of your rating struggles on C, and the system compensates by giving you a smaller penalty for missing C. Other rounds have an unusually easy C, which means solving it is essentially required to maintain your rating. You cannot predict this in advance, but you can recognise it during the contest: if 30 minutes have passed and the C solve count on the scoreboard is much higher or lower than usual for that position, adjust your strategy accordingly. In an easy-C round, prioritise speed on C; in a hard-C round, prioritise a clean B and a hack attempt on B.

The Role of Contest Frequency

How often should you compete? The answer depends on how much you upsolve. If you compete in three contests per week but never up-solve, you will plateau — you are exposing yourself to new problems but not learning from them. If you compete in one contest per week and up-solve two problems from it, you will climb steadily. The sweet spot for most climbers in the 1300-1600 range is two contests per week, with at least one up-solved problem per contest, plus 6 to 8 practice problems on non-contest days. This is roughly 10-15 hours of practice per week, which is what the six-month plan in Part VI assumes.

There is also a minimum frequency below which progress stalls. If you compete less than once per week, your contest instincts — the ability to read a problem, classify it, and decide whether to attempt it — decay faster than your underlying knowledge grows. The result is that you come back to contests feeling rusty and make tactical errors that cost you rating. If life forces you into a low-frequency period, prioritise maintaining contest frequency over practice volume — even one virtual contest per week is enough to keep the instincts sharp.

> *Your rating is a relative measure, not an absolute one. Climbing means beating the prediction, which means beating other people of your rating, which means solving problems they are not solving. The techniques in this guide are the ones that consistently separate climbers from stagnators at the 1300-1600 band.*

Chapter 2 — Self-Assessment and Goal Setting

KNOW EXACTLY WHERE YOU ARE BEFORE YOU CLIMB

The biggest mistake climbers make at the 1300-1500 band is practising without a clear picture of where they actually are. They open the problem set, sort by difficulty, and grind — but they do not track which topics they are weak in, which types of problems they consistently miss, or how their speed compares to what is needed at the next rating band. The result is hundreds of hours of practice that produces a slow, random walk up the rating ladder instead of a deliberate climb. This chapter fixes that.

The Diagnostic

Before you start the six-month plan, spend one week — about 10 to 12 hours — running a diagnostic on yourself. The diagnostic has three parts. First, solve 10 problems at rating 1300 across the major topic categories: greedy, binary search, implementation, math, constructive, and basic data structures. Time yourself honestly. Second, solve 10 problems at rating 1500 across the same categories. Third, attempt 5 problems at rating 1700 — these will mostly be failures, and that is the point. Record for every problem: time to solve (or time to give up), topic, whether you needed the editorial, and one sentence on what you learned or what you missed.

After the diagnostic, you will have a clear picture of three things. Your speed baseline: how long a 1300 problem takes you when you solve it cleanly, and how much longer a 1500 problem takes. Your topic weaknesses: which categories produced the most missed problems and the most editorial-lookups. Your ceiling: at what rating you start missing problems you cannot recover even with the editorial. Write these three numbers down. They are your starting coordinates. The six-month plan is calibrated to move all three: speed should drop by 30-40%, weaknesses should consolidate into 1-2 topics by month 4, and your ceiling should rise by at least 200 rating points.

The Rating Ladder

Breaking the climb into milestones makes it mentally manageable and lets you adjust the plan if you fall behind. Here is the ladder from 1300 to 1600, with what each milestone requires:

Milestone	Rating	What It Requires	Typical Time
1300 → 1400	Specialist entry	Consistent A+B+C solves; clean implementation; basic greedy and binary search fluency.	4-6 weeks
1400 → 1500	Mid-Specialist	C solves under 40 minutes; first DP and graph-pattern recognition; speed on A and B.	6-8 weeks
1500 → 1600	Expert entry	D solves in ~30% of rounds; deep DP/graph fluency; hack attempts; clean contest tactics.	6-10 weeks

Table 2.1 — The three-milestone ladder from contest rank 4000-5000 to Expert.

These time estimates assume 10-15 hours of focused practice per week and at least one rated contest per week. If you can only commit 5-8 hours per week, double the time estimates. If you can commit 20+ hours per week, you can compress the schedule by about 30%. The point is not to hit the milestones on a fixed calendar — it is

to make sure each milestone is achieved with the underlying skills actually in place, not skipped over. Reaching 1500 without solid DP fluency will leave you stuck at 1500 for months; better to spend an extra two weeks at 1400 cementing the DP foundations than to rush forward and stall.

Tracking Progress

There are three numbers worth tracking over the six months. The first is your displayed rating — the obvious one, but also the noisiest. Look at it weekly, not daily. The second is your virtual contest rating — run a virtual contest twice a week at roughly your target difficulty, and track the rating it produces. Virtual contests remove the psychological pressure of real ones and give a cleaner signal of underlying skill. The third is your upsolve ratio: of the problems you attempted but did not solve in the contest, what fraction did you eventually up-solve? A healthy upsolve ratio is 60% or higher. If it drops below 40%, you are either attempting problems too far above your level or skipping the learning phase — both are warning signs.

Maintain a simple log: a spreadsheet or a text file with one row per contest and one row per practice session. Columns: date, problem ID, rating, topic, time-to-solve, solved or not, used editorial or not, one-line takeaway. After six months you will have a rich dataset that shows exactly which topics accelerated your climb and which held you back. Most climbers skip this step and then have no idea why they stalled. Do not be most climbers.

Setting SMART Goals

Vague goals like "get better at CP" or "reach Expert" do not drive action. Convert your goal into SMART form: Specific, Measurable, Achievable, Relevant, Time-bound. A good SMART goal for the start of this plan looks like: "Solve 30 problems rated 1500-1600 on DP and graphs over the next 8 weeks, with an upsolve rate of 60% or higher, to reach a virtual contest rating of 1500 by the end of week 8." That goal tells you exactly what to do each week, gives you a number to check, is achievable on 12 hours per week, is relevant to your climb, and has a deadline. Re-set the SMART goal every 4 weeks based on where you actually are, not where the plan said you should be.

> *Practice without tracking is just typing. Track three numbers — displayed rating, virtual rating, upsolve ratio — and adjust the plan based on what they tell you, not on how you feel about your progress.*

Chapter 3 — Mindset and Psychology of Rating Climbing

THE MENTAL GAME THAT DECIDES WHO BREAKS THROUGH

Technical skill is necessary but not sufficient for climbing out of the 1300-1500 band. The climbers who break through to Expert and stay there share a set of mental habits that the stagnators do not. These habits are not innate — they can be learned, and most of them come down to managing your relationship with the rating number itself. If you treat every contest as a referendum on your worth as a programmer, you will burn out before you reach 1600. If you treat each contest as a data point in a long experiment, you will keep climbing through the inevitable plateaus.

Tilt Management

Tilt is the emotional state that follows a bad contest — frustration, self-doubt, and a strong urge to either quit or immediately grind more problems to "prove" yourself. Both responses are destructive. Quitting obviously stops your climb. Grinding more problems in a tilted state produces shallow, rushed work that does not stick, and it reinforces the emotional roller-coaster that makes the next bad contest even more tilting. The fix is simple to state and hard to execute: after a bad contest, do not look at problems for 24 hours. Read the editorial the next day, do the upsolve calmly, and only then return to regular practice.

A useful reframe: a bad contest is not a signal that you have gotten worse. It is a signal that the specific problem set on that specific day did not match your specific strengths. Over a large enough sample, contests average out — your rating will reflect your true skill within about 50 points after 30 contests. Single-contest swings of ±80 are normal and mean nothing. Track your rolling 5-contest average, not your last delta, and the tilt will shrink dramatically.

Rating Anxiety

Many climbers develop a fear of competing: they worry that a bad contest will drop their rating, so they avoid rated rounds and stick to practice. This is counterproductive — practice without contests does not build contest instincts, and contests are the only thing that actually moves your rating. The way out is to compete enough that any single contest stops feeling high-stakes. After 50 contests, a -40 delta feels like nothing; after 5 contests, it feels catastrophic. Get to 50 contests as fast as you reasonably can, even if some of them are virtual.

A related trap is "saving" your rating by not competing when you feel unprepared. This almost always backfires. The unpreparedness feeling is rarely about actual skill gaps — it is usually about fear of losing the rating you have. But ratings decay in spirit even when they do not decay in number: a 1500 rating you have not defended in 3 weeks is softer than a 1450 you have been actively defending. Compete regularly, accept the volatility, and trust the long-run average.

Focusing on Skills, Not Numbers

The healthiest mental frame for the six-month climb is to treat the rating as a lagging indicator of underlying skills. You do not directly control your rating — you control how many problems you solve, how fast you solve them, and how much you up-solve. The rating is the system's response to those inputs over time. If you focus on the inputs — solving 2 more problems per week than last month, dropping your average A-solve time by 2 minutes, raising your upsolve ratio from 50% to 70% — the rating will follow. If you focus on the rating directly, you will make tactical errors like avoiding hard contests or skipping problems you might fail on, both of which cap your learning.

Concretely: pick 2 or 3 process goals for each month, not outcome goals. A process goal is something you control directly — "solve 30 problems at 1500+ this month", "up-solve every C I miss for the next 4 weeks", "run a virtual contest every Saturday and Sunday". An outcome goal is something you do not directly control — "gain 100 rating this month". Process goals drive consistent action; outcome goals create anxiety when they are missed and overconfidence when they are exceeded.

Plateau Periods

Almost every climber hits at least one plateau during the six months — a 3-6 week period where the rating barely moves despite continued practice. Plateaus are normal, they are not a sign that you have stopped learning, and they almost always break suddenly when they do break. The reason is that skill acquisition is not linear: you spend weeks building pattern recognition that does not show up in contest results until enough patterns have accumulated to actually change which problems you can solve. Then the rating jumps 50-100 points in 2 weeks.

When you hit a plateau, do not change the plan. Keep doing the same practice, keep competing, keep upsolving. The only adjustment worth making is to widen your topic range — if you have been focused on DP for 4 weeks, spend a week on graphs or greedy instead. The variety sometimes unlocks the missing pattern. But do not abandon the plan or triple your practice volume — both produce burnout, not breakthroughs.

Building Resilience

Resilience after bad contests is a skill, not a trait. The climbers who reach Expert and stay there have a routine for processing bad contests: they wait 24 hours, read the editorial carefully, up-solve at least one problem they missed, write a one-line takeaway in their log, and then move on. They do not re-hash the contest mentally for days, they do not compare themselves to peers who did better, and they do not change their plan based on a single data point. This routine takes practice to internalise, but it is one of the highest-leverage mental skills in competitive programming.

> *Your rating over 30 contests reflects your true skill. Your rating over 1 contest reflects noise. Plan, practise, and judge yourself on the long-run average, not the latest data point.*

Chapter 4 — Setting Up Your Practice Environment

REMOVE EVERY SOURCE OF UNNECESSARY FRICTION

A bad practice environment costs you 10-20% of your effective practice time. Over six months at 12 hours per week, that is 30-60 hours wasted on friction — compiling, finding test cases, fixing preventable bugs, navigating the Codeforces interface inefficiently. This chapter walks through a setup that eliminates the common friction sources so that your practice hours actually translate into learning. Spend one afternoon setting this up; it pays for itself within the first week.

C++ Compiler and Flags

Codeforces uses the GNU C++20 compiler, so you should be testing locally with the same. On Linux, install g++ from your package manager (typically g++-12 or newer). On macOS, install gcc via Homebrew — the default Apple clang is close but not identical, and the differences will bite you on edge cases. On Windows, install MinGW-w64 or use WSL2 with a Linux g++. Whatever platform you use, verify that the version you compile with locally is within one minor version of what Codeforces uses; check the compiler version on the Codeforces submission page if you are unsure.

Use the same compile flags Codeforces uses for the C++20 (64) submission option. The flags are: -O2 -std=c++20 -Wall -Wextra -Wshadow. The -O2 enables optimisations so local timing approximates contest

timing. The -std=c++20 matches the contest language version. The -Wall -Wextra -Wextra flags turn on warnings that catch a large fraction of silly bugs before you ever submit. The -Wshadow flag warns when a local variable shadows an outer variable — a notoriously common source of subtle bugs.

Template File Structure

Every contest problem should start from the same template. The template handles fast I/O, common macros, and debug output that you can toggle on and off. A minimal but complete template looks like this:

```
#include <bits/stdc++.h>
using namespace std;

#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
#define rep(i, a, b) for (int i = (a); i < (b); i++)
#define per(i, a, b) for (int i = (b)-1; i >= (a); i--)
using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;
using vll = vector<ll>;

void solve() {
    int n;
    cin >> n;
    // ...
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;
    while (t--) solve();
    return 0;
}
```

The ios::sync_with_stdio(false) and cin.tie(nullptr) lines speed up input and output by a factor of 3-5x, which matters on problems with large inputs. The macros save typing and reduce off-by-one errors. The using declarations give you short names for common types. The bits/stdc++.h header includes the entire standard library — never use it in production code, but in contests it saves you from remembering which header a particular container or algorithm lives in.

Debug Macro

A debug macro lets you print variables to stderr during local testing, and the prints disappear automatically when you submit to Codeforces. The standard pattern is:

```
#ifdef LOCAL
#define dbg(...) cerr << "[" << #__VA_ARGS__ << "]: "; \
  _dbg(__VA_ARGS__)
template <typename T, typename... Args>
void _dbg(T first, Args... rest) { cerr << first; if constexpr (sizeof...(rest)) { cerr << ", ";
_dbg(rest...); } cerr << "\n"; }
```

```
#else
#define dbg(...)
#endif
```

Compile locally with -DLOCAL to enable the debug output; submit without -DLOCAL and the dbg() calls become no-ops. This pattern means you never have to remove debug prints before submitting, which removes a common source of submission errors.

Stress Testing

A stress test is the most powerful debugging tool for problems where you have a brute-force solution and a faster solution that you suspect has a bug. The pattern is: write the brute force, write the fast solution, generate random small test cases, run both, and compare outputs. When they disagree, you have found a bug. This is more reliable than trying to construct edge cases by hand, because the random generator will eventually hit the case your intuition missed.

A simple stress test harness looks like this:

```
int main() {
    for (int it = 0; it < 10000; it++) {
        // generate random input
        int n = rand() % 10 + 1;
        vector<int> a(n);
        for (auto &x : a) x = rand() % 20 - 10;

        // run both solutions
        auto brute = solve_brute(n, a);
        auto fast = solve_fast(n, a);

        if (brute != fast) {
            cout << "MISMATCH on test " << it << "\n";
            cout << "n=" << n << " a="; for (int x : a) cout << x << " "; cout << "\n";
            cout << "brute=" << brute << " fast=" << fast << "\n";
            return 0;
        }
    }
    cout << "All tests passed\n";
    return 0;
}
```

Run stress tests whenever you have a brute-force solution available — usually the brute force is obvious for n small, and writing it takes 5-10 minutes. The stress test will catch bugs that would otherwise cost you 30+ minutes of manual debugging during a contest.

Editor and Workflow

Use whatever editor you are fastest in — VS Code, CLion, Vim, Neovim, Emacs all work. The key requirement is that you can compile, run, and test with a single keystroke, and that you have a fast way to test against multiple input files. Set up a build task that compiles with the flags above and runs the current file. Set up a snippet for your template so that starting a new problem is one keystroke. Set up a key binding to run the current file against input.txt and write output to output.txt — this lets you pipe test cases without typing them into the terminal every time.

Beyond the editor, set up a clean folder structure for practice. One folder per problem, named with the problem ID (e.g., 1772D.cpp). Inside each folder: the solution file, an input.txt with the test case, and optionally a brute.cpp if you wrote one. This sounds like overkill but it pays off: six months in, you will have 200+ solved problems, and being able to grep through them for "how did I solve that DP problem three months ago" is invaluable.

Codeforces Interface Tips

Three Codeforces interface habits save real time. First, use the "Problemset" page rather than searching for problems by contest — the problemset lets you filter by rating and tag, which is how you should be selecting practice problems. Second, when reading editorials, open the editorial in one tab and the submissions of high-rated users in another — reading a clean implementation of the editorial often clarifies ideas that the editorial text left ambiguous. Third, use the "Status" page filtered by problem ID and rating to find short, clean submissions to study — filter for "C++20" and "Accepted" and sort by submission time to find the fastest clean solutions.

> *A 30-minute investment in your practice environment on day one saves 30+ hours over the six-month plan. Set it up properly before you start week 1.*

PART II

Core Skills

The foundational toolkit every climber needs before tackling DP and graphs: time complexity analysis, the C++ standard library, bug-free implementation, greedy thinking, binary search, and two-pointer techniques. Master these first.

Chapter 5 — Time Complexity and Big-O Analysis

THE SINGLE SKILL THAT PREVENTS 80% OF TLE FAILURES

Time complexity is the single most important analytical skill in competitive programming. A solution that compiles, passes samples, and produces correct output on every test case will still fail if it takes too long. The Codeforces time limit — typically 1 or 2 seconds — is not negotiable, and the difference between a passing O(n log n) solution and a failing O(n^2) solution is often a single character in your code. This chapter teaches you to look at a problem, look at its constraints, and immediately know what time complexity is required. That skill alone will save you from attempting impossible approaches and from wasting contest time on solutions that cannot possibly work.

The 10^8 Rule of Thumb

A modern CPU running a single thread with -O2 optimisation can perform roughly 10^8 simple operations per second. This is the number that every complexity calculation in competitive programming is based on. If your solution performs at most 10^8 operations, it will almost certainly run within a 1-second time limit. If it performs 10^9 operations, it will almost certainly time out. The range 10^8 to 10^9 is a grey zone where the actual runtime depends on the type of operation — simple integer arithmetic fits in 10^8, but expensive operations like random memory access or modulo division might fit only 3-5x10^7 in the same time. When in doubt, target 10^8 or fewer.

In practice, this means: given the input size n, your solution must perform at most 10^8 operations. Working backwards from this constraint tells you the maximum allowable complexity. The table below maps common input sizes to the complexities that fit within the 10^8 budget. Memorise this table — every DP and graph decision you make in contests depends on it.

Input Size n	Max Complexity	Typical Algorithm	Typical Problem Rating
n ≤ 20	O(2^n · n) or O(n!)	Bitmask DP, brute force permutations	1200-1500
n ≤ 100	O(n^3)	Floyd-Warshall, naive matrix ops	1400-1700
n ≤ 1000	O(n^2)	Naive DP, nested loops with break	1300-1600
n ≤ 10^5	O(n log n)	Sorting, segment tree, BIT, binary search	1400-1900
n ≤ 10^6	O(n) or O(n log log n)	Linear scan, sieve, two pointers	1500-2000
n ≤ 10^9	O(log n) or O(1)	Binary search on answer, math formula	1600-2200

Table 5.1 — Input size to maximum allowable complexity. Memorise this table.

The way to use this table during a contest is to read the problem, identify the largest input size from the constraints section, and immediately rule out any algorithm whose complexity exceeds the corresponding row. If the constraint says n ≤ 10^5, an O(n^2) solution will perform 10^10 operations — two orders of magnitude over budget — and you should not even consider writing it. If the constraint says n ≤ 1000, an O(n^2) solution

performs 10^6 operations, well within budget, and you can confidently write the nested loop. This single habit — checking the constraint against the complexity table before starting to code — prevents the most common cause of wasted contest time.

Counting Operations Accurately

Big-O notation hides constant factors, and in competitive programming those constant factors can matter. A solution that is O(n log n) with a small constant — like sorting with std::sort — runs much faster than an O(n log n) solution with a large constant — like repeatedly calling a hash function. When you are calculating whether your solution fits within the time limit, you need to account for the constant factor in addition to the asymptotic complexity. A rough rule: std::sort on 10^6 integers takes about 100ms; a segment tree update on 10^6 elements takes about 200-300ms; a hash table lookup on 10^6 elements takes about 50-100ms. Build this intuition by timing your solutions locally on maximum-size inputs.

A common trap is the "hidden log factor" — an algorithm that looks O(n) but is actually O(n log n) because of an internal data structure. For example, using std::map inside a loop that runs n times gives O(n log n) total, not O(n). Using std::unordered_map gives O(n) on average but O(n^2) in the worst case if an adversary crafts inputs that trigger hash collisions — a real attack that has been used in Codeforces contests. When you need O(1) lookups in a contest, prefer std::unordered_map with a custom hash (e.g., xor the key with a random constant) or use a sorted vector with std::lower_bound for O(log n) lookups that are not vulnerable to attack.

Common Complexity Traps

The first trap is sorting inside a loop. If you have a loop that runs n times and inside it you sort an array of size n, the total complexity is O(n^2 log n) — not O(n log n). The fix is usually to sort once before the loop and then use the sorted structure inside the loop. The second trap is using std::map or std::set for what should be array access. A map lookup is O(log n); an array access is O(1). If your keys are small integers (say, less than 10^7), use an array, not a map. The performance difference on 10^6 operations is roughly 10x — often the difference between passing and TLE.

The third trap is the "off-by-one in the loop bound" that turns an O(n) algorithm into O(n^2). For example, a substring search that uses two nested loops where the inner loop runs from i to n instead of i to i+k will run in O(n^2) instead of O(nk). Always check loop bounds against the intended complexity. The fourth trap is recursion depth — a recursive DFS on a chain of 10^5 nodes will stack overflow unless you increase the stack size or convert to an iterative version. Codeforces default stack size is generous, but if your solution recurses deeper than about 10^5 frames, you should switch to an explicit stack or use iterative traversal.

Amortised Analysis

Some algorithms have operations that individually look expensive but are cheap on average. The most common example is the std::vector push_back: a single push_back that triggers a resize is O(n), but a sequence of n push_back operations is O(n) total — amortised O(1) per operation. This is why you should almost never pre-allocate a vector to its exact size unless you know the size in advance; the amortised cost is the same and the code is simpler. The same amortised argument applies to DSU with path compression (each operation is amortised O(alpha(n)), essentially O(1)) and to the two-pointer technique (each pointer moves at most n times total).

When analysing your own solutions, look for patterns where a pointer or index only moves forward, never backward — these are usually amortised O(n) over the whole loop, not O(n^2) as a naive analysis would suggest. This insight is the foundation of the two-pointer and sliding-window techniques covered in Chapter 10.

> *Before writing any solution, check the input constraints against the complexity table. If your planned complexity does not fit, do not start coding — find a different approach.*

Chapter 6 — C++ STL for Competitive Programming

THE STANDARD LIBRARY IS YOUR SECOND BRAIN

The C++ Standard Template Library is the single biggest advantage C++ has over other languages in competitive programming. A well-chosen STL container or algorithm can replace 20-50 lines of hand-written code, and the STL implementations are heavily optimised — they will almost always be faster than what you would write yourself in the time available. This chapter covers the STL components that show up most often in 1300-1600 problems, with notes on when to use each, common pitfalls, and performance characteristics you need to know.

Containers

The workhorse containers for competitive programming are vector, set, map, priority_queue, and the adapters stack and queue. You should be using vector for at least 80% of your sequence storage needs — it has the best cache performance, the simplest API, and the fastest iteration. Use set when you need ordered unique elements with O(log n) insertion, deletion, and lookup. Use map when you need an ordered key-value mapping with the same complexities. Use priority_queue when you need a max-heap (or min-heap, with greater) and you only ever access the top element.

Container	Use When	Complexity	Common Pitfall
vector	Sequence storage; random access	O(1) access, O(1) amortised push_back	Iterator invalidation on resize
set / multiset	Ordered unique elements; find successor	O(log n) all ops	Cannot modify elements in place — erase + re-insert
map / multimap	Ordered key-value pairs	O(log n) all ops	operator[] default-constructs if key missing
unordered_set / unordered_map	Fast O(1) average lookup	O(1) avg, O(n) worst	Hash collision attack — use custom hash
priority_queue	Max-heap; Dijkstra; event processing	O(log n) push/pop, O(1) top	No decrease-key; cannot iterate in order
deque	Front and back push/pop; 0-1 BFS	O(1) front/back, O(n) middle	Slower than vector for random access
stack	LIFO; DFS explicit; bracket matching	O(1) top, push, pop	No iteration; use vector if you need it

Container	Use When	Complexity	Common Pitfall
queue	FIFO; BFS	O(1) front, back, push, pop	No random access

Table 6.1 — STL containers and their competitive programming use cases.

Algorithms

The STL algorithms live in and are essential for clean, fast code. The most frequently used are sort, lower_bound, upper_bound, unique, nth_element, and the heap operations. std::sort is O(n log n) and is essentially always faster than any hand-written sort you would write in a contest. lower_bound and upper_bound give O(log n) search on sorted ranges — use them instead of writing your own binary search, unless the binary search is on the answer (see Chapter 9). unique removes consecutive duplicates from a sorted range, which is the standard way to coordinate-compaction a set of values.

A common pattern that combines several STL algorithms is coordinate compression. Given an array of values that you want to use as indices but whose values are too large (say, up to 10^9), compress them to the range [0, n) with this idiom:

```
vector<long long> a = {/* ... */};
vector<long long> b = a;
sort(all(b));
b.erase(unique(all(b)), b.end());
// Now b[i] is the i-th smallest unique value.
// To compress a value v, find its index:
int idx = lower_bound(all(b), v) - b.begin();
```

This pattern shows up in segment tree problems, Fenwick tree problems, and any problem where you need to use values as array indices but the values are sparse. Memorise it.

When to Use Which Container

A common decision point is between map and unordered_map. The rule of thumb: use map when you need ordered iteration or when you want to be safe from hash attacks, use unordered_map when raw lookup speed matters more than ordering and you are sure the test data will not be adversarial. In practice, on Codeforces, you should assume the test data might be adversarial — there is a long history of problems where the standard unordered_map hash was exploitable. Either use map (safe, slightly slower) or use unordered_map with a custom hash that XORs the key with a random constant.

Another common decision is between set and sorted vector. If you build the structure once and then only query it, a sorted vector with lower_bound is faster than a set — the vector has better cache locality, and lower_bound is O(log n) just like set::find. If you need to interleave insertions and queries, set is the right choice. If you need to remove elements and re-add them, sorted vector becomes painful (O(n) per removal) and set is clearly better.

Common STL Bugs

The most common STL bug is iterator invalidation. When you modify a vector (push_back, insert, erase), all iterators into it may become invalid. If you are iterating over a vector and modifying it in the same loop, you will get undefined behaviour. The fix is usually to use index-based iteration instead of iterator-based, or to

collect the changes and apply them after the loop. The second most common bug is using operator[] on a map to check if a key exists — operator[] default-constructs the value if the key is missing, which silently inserts a zero-valued entry. Use map::count or map::find for existence checks.

The third common bug is the unordered_map hash attack mentioned above. The fourth is forgetting that priority_queue is a max-heap by default — to get a min-heap, declare it as priority_queue, greater>. The fifth is using set::lower_bound when you should be using the free function lower_bound on a sorted vector — the latter is faster due to cache locality, even though both are O(log n) asymptotically. These five bugs account for the majority of STL-related contest failures; knowing them by name will save you hours of debugging.

Built-in Functions

GCC provides a family of built-in functions that are extremely useful in competitive programming and are available on Codeforces. The most common are __builtin_popcount (count the number of set bits in an integer), __builtin_ctz (count trailing zeros), __builtin_clz (count leading zeros), and __lg (integer log base 2). These are O(1) on modern hardware and are the standard way to do bit operations in C++20. For 64-bit integers, use the long long variants: __builtin_popcountll, __builtin_ctzll, and so on.

```
int x = 42; // binary 101010
int bits = __builtin_popcount(x); // 3
int tz = __builtin_ctz(x); // 1 (one trailing zero)
int lz = __builtin_clz(x); // 26 (26 leading zeros in 32-bit)
int log2_x = __lg(x); // 5 (floor(log2(42)) = 5)
int lowest_bit = x & -x; // 2 (lowest set bit value)
int clear_lowest = x & (x - 1); // 40 (clear lowest set bit)
int subset = (x - 1) & x; // see bitmask DP chapter
```

Use the STL aggressively. Every line of STL you write is a line you did not have to debug, and the STL is faster than what you would write yourself in the time available.

Chapter 7 — Implementation and Bug-Free Coding

THE UNGLAMOROUS SKILL THAT WINS CONTESTS

Implementation skill is the ability to translate a correct algorithmic idea into working code, quickly, without bugs, on the first attempt. At the 1300-1600 band, this skill is often the difference between gaining and losing rating. Many climbers have the algorithmic knowledge to solve a 1500-rated problem but lose 20 minutes to a bug, miss the C solve, and end up with a worse rating than they started with. This chapter is about closing that gap — turning your algorithmic knowledge into contest results by writing cleaner code, catching bugs before submission, and debugging efficiently when bugs do slip through.

Translating Ideas to Code

The biggest implementation mistake at this level is starting to code before the algorithm is fully specified. You have a vague idea, you start writing, and you discover halfway through that the idea does not quite work or has a corner case you did not think through. Now you have 30 lines of code that need to be reworked, and the clock is ticking. The fix is to spend 3-5 minutes writing out the algorithm in pseudocode or plain English

before touching the keyboard. Specify the inputs, the outputs, the data structures, the loop structure, and the corner cases. If you cannot write the algorithm clearly in pseudocode, you are not ready to code it — go back to the drawing board.

A useful template for the pre-coding phase is the four-line summary: (1) what does the algorithm compute, (2) what is the main data structure, (3) what is the main loop, (4) what are the corner cases. If you can answer these four questions in one sentence each, you are ready to code. If you stumble on any of them, you need more thinking time. This discipline feels slow at first, but it actually speeds you up because you avoid the code-rewrite cycle that eats contest minutes.

Defensive Coding

Defensive coding means writing code that fails loudly when something goes wrong, rather than silently producing wrong output. In competitive programming, this means using assertions liberally, checking array bounds, and validating input assumptions. An assertion that fires during local testing tells you exactly where the bug is; an assertion that never fires tells you that path was not exercised and you need a different test case. Use assertions for invariants — conditions that should always hold at a given point in the code. For example, in a binary search, assert that the search range is non-empty at the start of each iteration; in a DP, assert that the state you are about to read has been computed.

A second defensive technique is to use signed integers everywhere unless you have a specific reason to use unsigned. Unsigned integer subtraction can wrap around to a huge positive number, which produces silently wrong results that are very hard to debug. The standard advice is to use int for indices when n ≤ 10^9 (int goes up to about 2.1 x 10^9) and long long for anything that might overflow int — including products of two ints, sums of n ints where n is large, and any computation involving n^2 or larger. When in doubt, use long long; the 2x memory and 2x time overhead is rarely the difference between AC and TLE, and the safety from overflow is worth it.

Common Bug Patterns

The four most common bug patterns at this level are off-by-one errors, integer overflow, uninitialised variables, and wrong loop bounds. Off-by-one errors typically appear in binary search (Chapter 9), in DP transitions where you access dp[i-1] without checking i > 0, and in slicing operations where you compute the wrong end index. The defence is to test boundary cases explicitly: if your loop runs from i = 0 to i < n, test with n = 0, n = 1, and n = 2 to make sure the boundary behaviour is correct.

Integer overflow typically appears when you multiply two values that fit in int but whose product does not. The classic case is computing n*(n-1)/2 for n up to 10^5 — n*(n-1) is about 10^10, which overflows int but fits in long long. The defence is to cast to long long before the multiplication: (long long)n * (n-1) / 2. The same applies to any computation involving powers, products, or sums of large arrays. As a rule of thumb, if the answer could plausibly exceed 10^9, use long long throughout the computation.

Uninitialised variables are the silent killer. A local variable that you forget to initialise will have whatever value happened to be in memory at that location, which means the bug appears only sometimes — depending on what ran before. The defence is to initialise every variable at declaration, even if you are sure you will write to it before reading. The compiler flag -Wmaybe-uninitialized (enabled by -Wextra) catches many of these, but

not all. The third bug, wrong loop bounds, is usually a typo — writing i <= n instead of i < n, or i < n-1 instead of i <= n-1. Code review against the algorithm specification catches these; coding without a specification makes them invisible.

Debugging Techniques

When a bug does slip through, the goal is to find it as fast as possible. The single most powerful debugging technique in competitive programming is the stress test described in Chapter 4 — write a brute force, generate random inputs, run both solutions, find the first mismatch. This works for any problem where a brute force exists, which at the 1300-1600 band is most of them. The stress test will pinpoint the exact input that breaks your solution, and from there the bug is usually obvious.

When stress testing is not available (because the brute force is too slow or because the output is not a single value), the next technique is binary search on the bug. Comment out half the code, run on the failing test, see if the bug is still there. If yes, the bug is in the remaining half; if no, the bug is in the commented half. Repeat until you have isolated the bug to a few lines. This is faster than reading the whole solution looking for the bug, because human eyes are bad at spotting bugs in code they just wrote. The third technique is the print-statement debug: add print statements at every key point in the code, run on the failing test, and trace through the output to find where the values diverge from what you expected. This is slower than stress testing but works on any bug.

Writing Short Clean Code

Shorter code has fewer bugs, is faster to write, and is easier to debug. The way to write shorter code is not to use terse syntax — it is to choose the right abstractions. If you find yourself writing the same 5 lines three times, factor it into a function. If you find yourself indexing into nested vectors repeatedly, introduce a reference: auto &row; = grid[i]; and then use row[j]. If you find yourself writing a 30-line solution to a problem that should be 10 lines, you are probably using the wrong algorithm — step back and think about whether there is a simpler approach.

A specific habit that produces cleaner code is to use std::array instead of C-style arrays, std::vector instead of new/delete, and std::string instead of char arrays. These containers manage their own memory, have bounds-checked access (with .at()), and integrate with the STL algorithms. The performance difference is negligible at this level, and the safety improvement is substantial. The only time you should use a C-style array is when you need a fixed-size global buffer that you want to avoid initialising on every test case — even then, std::array with the right initialisation is usually fine.

> *Five minutes of pseudocode before coding saves twenty minutes of debugging after. Specify the algorithm before you write it, and the bugs will not have anywhere to hide.*

Chapter 8 — Greedy Algorithms

WHEN THE LOCALLY OPTIMAL CHOICE IS GLOBALLY OPTIMAL

A greedy algorithm makes the locally optimal choice at each step, without backtracking, and hopes that this leads to a globally optimal solution. Greedy algorithms are attractive because they are usually simple to implement and fast to run, but they are dangerous because they are often wrong — the locally optimal choice

can paint you into a corner where the global optimum is unreachable. This chapter covers when greedy works, how to prove it works, and the classic greedy patterns that appear most often at the 1300-1600 difficulty band.

When Greedy Works

Greedy works when the problem has two properties: optimal substructure (the optimal solution to the problem contains optimal solutions to subproblems) and the greedy choice property (the locally optimal choice is always part of some globally optimal solution). These properties are not always obvious from the problem statement — usually you have to argue them from the structure of the problem. The standard technique for arguing the greedy choice property is the exchange argument: assume an optimal solution that does not make the greedy choice, and show that you can exchange the non-greedy choice for the greedy choice without making the solution worse.

As a concrete example, consider the activity selection problem: given n activities with start and end times, select the maximum number of non-overlapping activities. The greedy choice is to always pick the activity that ends earliest. The exchange argument: suppose an optimal solution does not include the earliest-ending activity. Then it includes some other activity that ends later. Replace that activity with the earliest-ending one — the replacement does not overlap any other activity in the solution (because the replacement ends earlier than what was there), so the solution is still valid and has the same size. Therefore the greedy choice is always part of some optimal solution. This argument is the template for most greedy proofs at this level.

Classic Greedy Patterns

The first pattern is sorting-based greedy. Many greedy problems reduce to: sort the input by some criterion, then process the sorted input and make a locally optimal choice at each step. The activity selection problem above is the canonical example. Other examples: scheduling jobs by deadline, picking tasks by reward, distributing resources by need. The sort criterion is usually obvious once you think about it — sort by end time for activity selection, by deadline for scheduling, by value-per-weight for fractional knapsack. If the sort criterion is not obvious, the problem probably is not a sorting-based greedy, or you have not understood the structure yet.

The second pattern is priority-queue greedy. Here you process elements in some order, but instead of always taking the next element, you maintain a priority queue of candidates and take the best one at each step. The canonical example is Huffman coding: at each step, take the two lowest-frequency symbols from the priority queue, merge them, and put the result back. Another common example is "process tasks with deadlines and rewards": sort by deadline, maintain a min-heap of selected task rewards, and at each step either add the current task or swap it with the smallest-reward selected task if the current is bigger. These problems are common at 1400-1700 difficulty.

The third pattern is exchange-argument greedy, where the greedy choice is not obvious but can be proven correct by exchange. The classic example is "minimum number of coins to make a value" with canonical coin systems — the greedy choice (always take the largest coin that fits) works for US coins but not for arbitrary coin systems. Proving that a coin system is canonical requires an exchange argument and is not always easy; in contests, if you are not sure, fall back to DP.

Common Pitfalls

The biggest pitfall is assuming greedy works without proof. A solution that "feels greedy" but is actually wrong will pass the sample tests, fail on system tests, and cost you 50 rating points. The defence is to always construct a counterexample before submitting. Take the greedy choice you are making, think about what input would make that choice wrong, and test it. If you cannot construct a counterexample after 2 minutes of trying, the greedy is probably right; if you can construct one, you need a different approach.

A specific pitfall at this level is greedy on a sorted input where the sort criterion is wrong. For example, in the "maximum number of tasks you can complete with deadlines and durations" problem, the correct sort is by deadline — but a common wrong sort is by duration. Both feel plausible, but only the deadline sort is correct. When you have a sorting-based greedy, always ask: "could sorting by a different criterion give a different answer, and if so, why is my criterion the right one?" If you cannot answer, you do not understand the problem well enough yet.

Practice Problems by Pattern

The way to internalise greedy thinking is to solve problems across all three patterns. For sorting-based greedy, work through problems tagged "greedy" at rating 1300-1500 — there are dozens on Codeforces. For priority-queue greedy, look for problems involving scheduling with deadlines, or problems where you need to select k items from n with some constraint. For exchange-argument greedy, the classic problems are coin change with canonical systems and various "minimum swaps" problems. Spend 2 weeks on greedy during month 1 of the plan, solving 8-10 problems across the patterns. After that, greedy will be a reliable tool in your contest arsenal rather than a hopeful guess.

> *Greedy without a proof is a guess. Always construct a counterexample before submitting — two minutes of counterexample hunting saves 50 rating points on a wrong submission.*

Chapter 9 — Binary Search and Ternary Search

THE MOST VERSATILE TECHNIQUE IN COMPETITIVE PROGRAMMING

Binary search is the single most versatile technique in competitive programming. It is used to search sorted arrays, to find roots of monotonic functions, to compute integer square roots, to optimise over a unimodal function, and — most importantly at the 1300-1600 band — to "binary search the answer" of an optimisation problem. The technique is simple to state but surprisingly hard to implement correctly; off-by-one errors in binary search are one of the most common sources of contest failures. This chapter covers the standard patterns and the common pitfalls.

Binary Search on a Sorted Array

The simplest form of binary search is finding an element in a sorted array. In C++20, you should never write this by hand — use std::lower_bound (returns iterator to first element >= target) or std::upper_bound (returns iterator to first element > target). Both are O(log n) and are essentially impossible to get wrong. The only reason to write a manual binary search is when the predicate is more complex than a simple comparison — for example, "is the element at position i valid according to some property?"

```
// Standard lower_bound usage:
#include <algorithm>
```

```
vector<int> a = {1, 2, 4, 4, 4, 7, 9, 11};

// Find first element >= 4:
auto it = lower_bound(a.begin(), a.end(), 4);
int idx = it - a.begin(); // idx = 2

// Find first element > 4:
auto it2 = upper_bound(a.begin(), a.end(), 4);
int idx2 = it2 - a.begin(); // idx2 = 5

// Count of elements equal to 4:
int count = upper_bound(a.begin(), a.end(), 4)
          - lower_bound(a.begin(), a.end(), 4); // 3
```

Binary Search on the Answer

The pattern that shows up most often at 1300-1600 is "binary search on the answer". The setup: the problem asks for the maximum (or minimum) value of some quantity X such that a certain property P(X) holds. The key insight is that P is monotonic — if P(X) holds for some X, then P(Y) also holds for all Y < X (for a maximisation problem) or all Y > X (for a minimisation problem). When P is monotonic, you can binary search over the range of possible X values, checking P(mid) at each step.

A typical example: "What is the maximum value k such that we can select k items from the array with sum at most S?" Here P(k) is "can we select k items with sum <= S", and P is monotonic because if we can select k items with sum <= S, we can certainly select k-1 items (just drop one). So we binary search on k. The check function P(k) is itself a sub-problem — usually a greedy or DP — but it is much simpler than the original problem because k is fixed.

The standard template for binary search on the answer looks like this:

```
bool check(long long x) {
    // Return true if x is achievable / valid.
    // Uses greedy, DP, or other technique.
}

long long lo = 0, hi = 1e18; // Adjust to problem-specific bounds
while (lo < hi) {
    long long mid = (lo + hi + 1) / 2; // +1 to avoid infinite loop on max
    if (check(mid)) {
        lo = mid; // mid is achievable, try higher
    } else {
        hi = mid - 1; // mid is not achievable, go lower
    }
}
// lo == hi is the maximum achievable value.
```

The mid = (lo + hi + 1) / 2 with lo = mid is the pattern for finding the maximum. For finding the minimum, use mid = (lo + hi) / 2 with hi = mid. The +1 in the maximum version is critical — without it, if lo and hi differ by 1, mid equals lo, and if check(mid) is true, lo = mid = lo produces an infinite loop. This is the most common binary search bug. Memorise the two patterns (max with +1, min without) and you will never have this bug again.

Ternary Search

Ternary search finds the maximum (or minimum) of a unimodal function — a function that increases then decreases (or vice versa). The idea is to evaluate the function at two interior points, discard the part of the range that cannot contain the optimum, and repeat. Unlike binary search, ternary search reduces the range by a factor of 2/3 per iteration, so it converges in O(log_{1.5} n) iterations.

```
// Find maximum of unimodal f on [lo, hi]
double lo = 0, hi = 1e9;
for (int i = 0; i < 200; i++) { // 200 iterations: precise enough for doubles
    double m1 = lo + (hi - lo) / 3;
    double m2 = hi - (hi - lo) / 3;
    if (f(m1) < f(m2)) {
        lo = m1; // maximum is in [m1, hi]
    } else {
        hi = m2; // maximum is in [lo, m2]
    }
}
// lo is approximately the maximiser
```

A common pitfall with ternary search on real numbers is using an epsilon-based stopping condition (while (hi - lo > 1e-9)) instead of a fixed iteration count. The epsilon approach can infinite-loop when the function is flat near the optimum, because hi - lo stops shrinking meaningfully. The fixed-iteration approach (200 iterations is plenty for any practical precision) avoids this entirely. Use the fixed-iteration version.

Monotonicity Recognition

The hardest part of binary search on the answer is recognising that the property is monotonic. The signal to look for is a phrase like "maximum value such that" or "minimum value such that" — these almost always indicate a binary-searchable answer. The next signal is a yes/no sub-problem hiding inside an optimisation problem: if you can phrase the problem as "for a given X, can we achieve X?", and the answer to that question is monotonic in X, then binary search applies. Spend time on this recognition — it is the difference between solving and not solving many 1400-1600 problems.

> *When a problem asks "maximum X such that property holds" and the property is monotonic in X, binary search the answer. The check function is usually simpler than the original problem — find it and the rest is mechanical.*

Chapter 10 — Two Pointers and Sliding Window

CONVERTING O(n^2) NAIVE SOLUTIONS TO O(n)

The two-pointer technique converts O(n^2) naive solutions into O(n) efficient ones for a specific class of problems involving sequences or arrays. The key insight is that when two pointers each only move in one direction (typically both forward), the total movement is at most 2n, which is O(n) — even though a naive analysis of the nested loop structure would suggest O(n^2). This chapter covers the standard two-pointer patterns and the sliding-window variant that appears in many 1300-1500 problems.

Two Pointers on Sorted Arrays

The simplest two-pointer pattern is on a sorted array. Given a sorted array, find pairs of elements that satisfy some property — typically a sum, difference, or product equal to a target. The naive approach is O(n^2) — try every pair. The two-pointer approach is O(n): place one pointer at the start and one at the end, compare the property of the pair to the target, and move the appropriate pointer based on the comparison. For a "two-sum" problem on a sorted array, if the current sum is less than the target, move the left pointer right (increasing the sum); if greater, move the right pointer left (decreasing the sum). Each comparison eliminates one element from consideration, so the total work is O(n).

```
// Two-sum on sorted array, return indices of pair summing to target
pair<int,int> two_sum(vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo < hi) {
        int sum = a[lo] + a[hi];
        if (sum == target) return {lo, hi};
        else if (sum < target) lo++;
        else hi--;
    }
    return {-1, -1}; // not found
}
```

Two Pointers on Sequences

A more general pattern is two pointers on a sequence where the pointers define a subarray. The classic problem is "longest subarray with sum at most K" or "shortest subarray with sum at least K". The pattern: maintain a window [lo, hi) and expand hi to add elements, contract lo to remove elements. The key invariant is that the window always satisfies the problem constraint, and you track the best window seen so far.

```
// Longest subarray with sum at most K
int longest_subarray(vector<int>& a, int K) {
    int n = a.size();
    int lo = 0, sum = 0, best = 0;
    for (int hi = 0; hi < n; hi++) {
        sum += a[hi];
        while (sum > K && lo <= hi) {
            sum -= a[lo];
            lo++;
        }
        best = max(best, hi - lo + 1);
    }
    return best;
}
```

The complexity is O(n) even though there is a while loop inside a for loop, because the lo pointer only moves forward across the entire algorithm — it moves at most n times total. This amortised analysis is the core of why two pointers works. Always check that both pointers only move in one direction; if they do, the algorithm is O(n) regardless of the nested loop structure.

Sliding Window: Fixed and Variable

A fixed-size sliding window is a special case where the window size is constant. The pattern: compute the value for the initial window, then slide the window by one position at a time — add the new element, remove the old element, update the value. This is O(n) for any window value that can be updated in O(1) given the

previous value. Common applications: maximum sum of any subarray of size k, count of distinct elements in any window of size k, maximum element in any window of size k (the last requires a deque, covered below).

A variable-size sliding window is the more general case where the window size changes based on the constraint. The "longest subarray with sum at most K" problem above is a variable-size window. The pattern is the same — expand hi, contract lo as needed — but the window size is not fixed. The complexity is still O(n) by the same amortised argument.

Monotonic Deque

A more advanced sliding-window technique is the monotonic deque, used to compute the maximum (or minimum) of every window of size k in O(n). The idea: maintain a deque of indices such that the corresponding values are decreasing (for a max-window). When a new element arrives, pop from the back of the deque any elements smaller than the new one (they can never be the max of any future window that includes the new element). Then push the new element. Pop from the front any element that has fallen out of the window. The front of the deque is the max of the current window.

```
// Max of every window of size k
vector<int> max_window(vector<int>& a, int k) {
    int n = a.size();
    vector<int> result;
    deque<int> dq; // stores indices, values decreasing
    for (int i = 0; i < n; i++) {
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        while (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) result.push_back(a[dq.front()]);
    }
    return result;
}
```

The complexity is O(n) because each element is pushed and popped at most once. The monotonic deque is a common 1500-1700 pattern, especially in problems involving range-minimum or range-maximum queries over a sliding window. Memorise the template above and you will be able to apply it without thinking during contests.

When Two Pointers Replaces Binary Search

Both two pointers and binary search can solve problems involving pairs or subarrays with a target property. Two pointers requires the array to be sorted (or to have a specific monotonic structure); binary search on the answer does not require sorting but does require the check function to be implementable efficiently. When both apply, two pointers is usually simpler and faster — O(n log n) for the sort plus O(n) for the search, versus O(n log C) for binary search on a range of size C. For sequence problems where the array is already sorted or where the natural order works, prefer two pointers.

> *Two pointers converts O(n^2) to O(n) when both pointers only move forward. Look for this pattern in any problem involving pairs, subarrays, or windows on a sorted or sortable sequence.*

PART III

Dynamic Programming Deep Dive

DP is the highest-leverage topic for climbing from Specialist to Expert. This part takes you from first principles through the patterns that show up most often at 1400-1800 difficulty, with extra attention to the state-design mental model that separates DP solvers from DP guessers.

Chapter 11 — DP Fundamentals

THE FOUR-STEP METHOD THAT MAKES DP APPROACHABLE

Dynamic programming is the technique of solving a problem by combining solutions to overlapping subproblems. It is the single most powerful algorithmic tool at the 1300-1800 difficulty band, and it is also the technique that climbers find most intimidating. The intimidation comes from treating DP as a black art — a set of clever tricks that you either see or you do not. This is the wrong frame. DP is a systematic method, and once you internalise the four-step recipe, most DP problems at this level become approachable. This chapter teaches the recipe; the next five chapters apply it to the major DP pattern families.

What DP Actually Is

A problem is solvable by DP when it has two properties. The first is optimal substructure: the optimal solution to the problem can be constructed from optimal solutions to its subproblems. The second is overlapping subproblems: the same subproblems appear many times in the recursion tree, so caching their solutions saves work. The first property is what makes the recursion valid; the second is what makes the caching worthwhile. If a problem has optimal substructure but no overlapping subproblems, you solve it with divide and conquer (like merge sort). If it has overlapping subproblems but no optimal substructure, DP gives the wrong answer. Both properties together is what makes DP the right tool.

A useful diagnostic: if you can phrase the problem as "the answer for input X is the best over all ways to split X into smaller pieces, of the (answer for the piece) combined with some local cost", then the problem likely has optimal substructure and DP applies. The phrase "best over all ways" is the key — it tells you the answer is a max or min of subproblem answers, which is the signature of DP.

The Four-Step Method

Every DP solution can be derived by following four steps in order. Skipping any step leads to the typical DP confusion — a vague idea that you cannot quite turn into code. Following the steps in order produces a working solution every time. The steps are: (1) define the state, (2) write the transition, (3) specify the base case, (4) identify the answer.

Step 1 — Define the state. The state is a description of a subproblem that is small enough to be uniquely identified by a few parameters. The state is what you memoize on. The art of DP is choosing the right state — too few parameters and you cannot distinguish subproblems; too many and the state space explodes. For most problems at this level, the state is a tuple of 1 to 3 integers. For example, in the longest increasing subsequence problem, the state is (i) — the length of the LIS ending at index i. In the knapsack problem, the state is (i, w) — the maximum value using the first i items with weight budget w. In edit distance, the state is (i, j) — the edit distance between the first i characters of string 1 and the first j characters of string 2.

Step 2 — Write the transition. The transition says: given the state, what other states does it depend on, and how is the answer for this state computed from those? For LIS ending at i, the transition is dp[i] = max(dp[j] + 1) over all j < i with a[j] < a[i]. For knapsack, dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight[i]] + value[i]). The transition is the heart of the DP — get it right and the rest follows. Get it wrong and no amount of implementation will save you. The way to derive the transition is to think about the "last decision": what is the final choice made in the subproblem, and how does that choice split the subproblem into smaller subproblems?

Step 3 — Specify the base case. The base case is the smallest subproblem that does not depend on any other subproblem — the recursion has to stop somewhere. For LIS, the base case is dp[i] = 1 for all i (every element alone is an increasing subsequence of length 1). For knapsack, dp[0][w] = 0 for all w (with no items, the best value is 0). For edit distance, dp[i][0] = i and dp[0][j] = j (editing to/from an empty string is just inserting/deleting all characters). Get the base case wrong and you get off-by-one errors or undefined behaviour when the recursion reaches the boundary.

Step 4 — Identify the answer. The answer to the original problem is some function of the states — usually a max, min, or sum over a set of states. For LIS, the answer is max(dp[i]) over all i. For knapsack, the answer is dp[n][W]. For edit distance, the answer is dp[n][m]. Identifying the answer correctly is critical: a common mistake is to compute the DP table correctly but then read off the wrong cell as the answer.

Top-Down vs Bottom-Up

There are two ways to implement a DP: top-down (memoised recursion) and bottom-up (iterative tabulation). Top-down starts from the answer state and recursively computes dependencies, caching results in a map or array. Bottom-up starts from the base cases and iteratively fills in the table in dependency order. Both produce the same answers and have the same asymptotic complexity, but they have different practical trade-offs.

Top-down is easier to write correctly because the transition is expressed as a recursive function that mirrors the mathematical definition. It only computes states that are actually needed, which is a real advantage when the state space is sparse. The downside is recursion overhead — each state lookup goes through a function call, which is slower than array access, and deep recursion can stack overflow. Bottom-up is faster (no recursion overhead, better cache locality) and avoids stack overflow, but it requires you to figure out the correct iteration order, which is not always trivial.

For contests, the standard advice is: use top-down when the state space is sparse or the transition is complex, use bottom-up when the state space is dense and the iteration order is obvious. At the 1300-1600 level, most problems have dense state spaces and obvious iteration orders, so bottom-up is usually the right choice. The exception is digit DP (Chapter 15), where the state space is sparse and the transitions are complex enough that top-down with memoization is the standard implementation.

Common State Dimensions

Most DP problems at this level have states with 1, 2, or 3 dimensions. One-dimensional DP appears in LIS, house robber, coin change (with one denomination), and various "linear scan with choice" problems. Two-dimensional DP appears in knapsack (item x weight), edit distance (i x j), longest common substring, and grid-path problems. Three-dimensional DP is rarer but shows up in problems like "two paths through a grid simultaneously" or knapsack with two budgets. The complexity of a DP is the product of the state space size and the transition cost, so a 2D DP with O(n) states and O(1) transition is O(n^2) total, while a 1D DP with O(n) states and O(n) transition is also O(n^2) total. Always compute this product before committing to a DP — if it exceeds 10^8, the DP will TLE.

> *DP is not a black art — it is a four-step method. Define the state, write the transition, specify the base case, identify the answer. Skip a step and you will be confused; follow the steps in order and most problems at this level become approachable.*

Chapter 12 — Classic DP Patterns

THE SIX PATTERNS THAT COVER 80% OF DP PROBLEMS AT THIS LEVEL

Most DP problems at the 1300-1700 difficulty band are variations on a small number of classic patterns. Once you internalise these patterns, recognising which one a new problem fits becomes second nature, and the implementation is mostly a matter of adapting the template. This chapter walks through the six most common patterns: Fibonacci-style, knapsack, longest increasing subsequence, longest common subsequence, edit distance, and coin change. For each, we give the state, the transition, the complexity, a code template, and the common variants you will see in contests.

Pattern 1: Fibonacci-Style 1D DP

The simplest DP pattern is the Fibonacci-style: dp[i] depends on a fixed set of previous states, typically dp[i-1] and dp[i-2]. The canonical example is "count the number of ways to climb a staircase of n steps, taking 1 or 2 steps at a time" — dp[i] = dp[i-1] + dp[i-2]. The state is just the index i, the transition is a sum of two fixed previous states, the base case is dp[0] = 1, dp[1] = 1, and the answer is dp[n]. Variants include tiling problems (2xN grid with 2x1 tiles), house robber (max sum with no two adjacent), and various "sequence with constraint" problems.

```
// House robber: max sum with no two adjacent elements
int rob(vector<int>& a) {
    int n = a.size();
    if (n == 0) return 0;
    if (n == 1) return a[0];
    vector<int> dp(n);
    dp[0] = a[0];
    dp[1] = max(a[0], a[1]);
    for (int i = 2; i < n; i++) {
        dp[i] = max(dp[i-1], dp[i-2] + a[i]);
    }
    return dp[n-1];
}
```

Pattern 2: 0/1 Knapsack

The 0/1 knapsack problem: given n items with weights and values, and a weight capacity W, select a subset with maximum total value such that the total weight is at most W. The state is (i, w) — the maximum value using the first i items with weight budget w. The transition is dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight[i]] + value[i]) — either skip item i or take it (if it fits). The base case is dp[0][w] = 0 for all w. The answer is dp[n][W]. The complexity is O(nW).

```
// 0/1 Knapsack — space-optimised to O(W)
int knapsack(int W, vector<int>& weight, vector<int>& value) {
    int n = weight.size();
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        // Iterate backwards to avoid using the same item twice
        for (int w = W; w >= weight[i]; w--) {
            dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);
        }
    }
```

```
    return dp[W];
}
```

A critical implementation detail: when space-optimising the knapsack to O(W), you must iterate the inner loop backwards. Iterating forwards would let the same item be used multiple times — which is actually the unbounded knapsack variant. For 0/1 knapsack (each item used at most once), iterate backwards; for unbounded knapsack (each item usable unlimited times), iterate forwards. Getting this backwards is a common bug.

Pattern 3: Longest Increasing Subsequence

The LIS problem: given an array, find the length of the longest strictly increasing subsequence. The naive O(n^2) DP is: dp[i] = max(dp[j] + 1) over all j < i with a[j] < a[i], answer is max(dp[i]). The O(n log n) version uses a different approach: maintain a vector tails where tails[k] is the smallest possible tail value of an increasing subsequence of length k+1. For each element, binary search to find the position to update. This is the version you should use in contests — it is faster and the code is short.

```
// LIS in O(n log n)
#include <algorithm>
int lis(vector<int>& a) {
    vector<int> tails;
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return tails.size();
}
```

A subtle point: the tails vector at the end is NOT necessarily a valid increasing subsequence — it is just the right length. To reconstruct an actual subsequence, you need to maintain a parent pointer array alongside the tails vector. For problems that just ask for the length, the simple version above is sufficient.

Pattern 4: Longest Common Subsequence

The LCS problem: given two strings, find the length of the longest subsequence common to both. The state is (i, j) — the LCS of the first i characters of string 1 and the first j characters of string 2. The transition: if s1[i-1] == s2[j-1], dp[i][j] = dp[i-1][j-1] + 1; otherwise dp[i][j] = max(dp[i-1][j], dp[i][j-1]). The base case is dp[0][j] = dp[i][0] = 0. The answer is dp[n][m]. The complexity is O(nm). This is the canonical 2D string DP and shows up in many variants — edit distance (next pattern), shortest common supersequence, distinct subsequences, and more.

Pattern 5: Edit Distance

The edit distance problem: given two strings, find the minimum number of single-character edits (insert, delete, replace) to transform one into the other. The state is (i, j) — the edit distance between the first i characters of s1 and the first j characters of s2. The transition: if s1[i-1] == s2[j-1], dp[i][j] = dp[i-1][j-1] (no edit needed); otherwise dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) (delete from s1, insert into s1, or replace). The base case is dp[i][0] = i, dp[0][j] = j. The answer is dp[n][m].

Pattern 6: Coin Change

The coin change problem: given a set of coin denominations and a target value, find the minimum number of coins to make the target (or the number of ways to make the target). The state is dp[v] — the answer for value v. The transition: dp[v] = min(dp[v - coin] + 1) over all coins with coin <= v, for the minimum-count version. The base case is dp[0] = 0. The answer is dp[V]. For the count-ways version, dp[v] = sum(dp[v - coin]) over all coins, with dp[0] = 1. Be careful about counting order: iterating coins in the outer loop counts combinations (order does not matter), while iterating value in the outer loop counts permutations (order matters). Most problems want combinations.

> *Most DP problems at this level are variations on six classic patterns: Fibonacci-style, knapsack, LIS, LCS, edit distance, coin change. Recognise the pattern, adapt the template, move on.*

Chapter 13 — DP on Trees

WHEN THE SUBPROBLEM STRUCTURE IS A TREE

Tree DP is DP where the subproblem structure is a tree — typically a rooted tree where each subtree is itself a subproblem. The state is usually a node (and possibly some additional information), the transition combines answers from children, and the answer is read off at the root. Tree DP shows up frequently at 1500-1800 difficulty and is one of the most rewarding patterns to master, because once you see the tree structure, the DP is usually mechanical. This chapter covers the standard patterns and the rerooting technique that extends single-root DP to all-roots DP.

The Basic Pattern

The basic tree DP pattern: pick an arbitrary root (usually node 0 or 1), do a post-order DFS, and at each node combine the DP values from its children into the DP value for the node. The state is the node, the transition is some combination (sum, max, min) of the children's states, and the base case is the leaves. The answer is read off at the root. The complexity is O(n) because each edge is processed exactly once during the DFS.

A canonical example: "find the diameter of a tree" (the longest path between any two nodes). The DP is: dp[v] = the length of the longest path starting at v and going down into its subtree. The transition: dp[v] = max(dp[child] + 1) over all children. But the diameter is not dp[root] — it is the max over all v of (sum of the two largest dp[child] values + 2), because the longest path through v goes down to two of its children. This is a common pattern in tree DP: the answer is not directly the root state, but some aggregation over all nodes.

```
// Tree diameter
vector<vector<int>> g;
int diameter = 0;

int dfs(int v, int parent) {
    int max1 = 0, max2 = 0; // two longest downward paths from v
    for (int u : g[v]) {
        if (u == parent) continue;
        int d = dfs(u, v) + 1;
        if (d > max1) { max2 = max1; max1 = d; }
        else if (d > max2) max2 = d;
    }
    diameter = max(diameter, max1 + max2);
```

```
    return max1;
}
```

The Rerooting Technique

A common tree DP problem asks for an answer at every node, not just at one root. For example: "for each node v, find the maximum distance from v to any other node". The naive approach is to run a BFS from every node — O(n^2) total. The rerooting technique does it in O(n). The idea: first run a standard tree DP from an arbitrary root to compute the answer at the root. Then do a second DFS where, when moving from a parent to a child, you "reroot" the tree at the child by updating the DP values to reflect the child being the new root. The update uses the parent's now-computed answer plus the contributions from the parent's other children.

The mechanics of rerooting are problem-specific but follow a template: at each node, you maintain the answer for the subtree rooted at that node. When moving to a child, you temporarily remove the child's contribution from the parent, set the parent's new "subtree answer" to the parent's rerooted answer (which includes everything except the child's subtree), then recurse into the child. After the recursion, restore the parent's contribution. The key insight is that the parent's rerooted answer is itself a tree DP answer that can be combined with the child's subtree answer to give the child's rerooted answer.

Common Tree DP Problems

The most common tree DP problems at this level are: tree diameter (covered above), maximum independent set on a tree (dp[v][0/1] = max independent set in v's subtree, with v either excluded or included), tree knapsack (select items attached to nodes with a budget that travels down the tree), count of paths with a given property (dp[v] aggregates paths through v), and distance sum (sum of distances from each node to all others — uses rerooting). Each of these has a standard template that you can find in any competitive programming reference; the skill is recognising which template applies to a new problem.

A common bug in tree DP is forgetting to handle the root specially. The root has no parent, so any transition that references "the parent contribution" needs a separate code path for the root. Another common bug is iterating over all nodes in the DFS instead of using the tree structure — this turns an O(n) DP into O(n^2) because each edge is processed multiple times. Always use the adjacency list and the parent parameter to ensure each edge is processed exactly once.

> *Tree DP is mechanical once you see the tree. Pick a root, do a post-order DFS, combine children into parent. For all-roots answers, use rerooting — O(n) instead of O(n^2).*

Chapter 14 — Bitmask DP

WHEN THE STATE IS A SUBSET

Bitmask DP is DP where one of the state parameters is a subset, represented as a bitmask (an integer where bit i is 1 iff element i is in the subset). Bitmask DP is the standard technique for problems involving permutations, subsets, or matchings on small sets — typically n ≤ 20, because the state space is 2^n and 2^20 is about 10^6. This chapter covers the core technique, the subset iteration trick, and the classic problems where bitmask DP appears.

Representing Subsets as Integers

A subset of {0, 1, ..., n-1} can be represented as an n-bit integer, where bit i is set iff element i is in the subset. The standard operations are: check if element i is in the set (mask & (1 << i)), add element i (mask | (1 << i)), remove element i (mask & ~(1 << i)), count the number of elements (__builtin_popcount(mask)), and iterate over all subsets of a mask (the (sub - 1) & mask trick below). These operations are all O(1) or O(n) and are the building blocks of bitmask DP.

The Subset Iteration Trick

A common need is to iterate over all subsets of a given mask. The naive approach is to iterate over all 2^n integers and check if each is a subset — O(2^n * n). The clever trick is to iterate only over the subsets, which is O(3^n) total across all masks but much faster for individual masks. The pattern is:

```
// Iterate over all subsets of `mask` (including empty, excluding `mask` itself)
for (int sub = (mask - 1) & mask; sub > 0; sub = (sub - 1) & mask) {
    // sub is a proper subset of mask
}

// To iterate over all subsets including mask itself:
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // sub is a subset of mask (including mask)
}
// Don't forget the empty subset if you need it
```

The trick is that (sub - 1) & mask gives the next-smaller subset of mask, in some order. This is a standard competitive programming idiom — memorise it. The total work across all subsets of all masks is O(3^n), which is feasible for n up to about 16-18.

Classic Bitmask DP Problems

The most common bitmask DP at this level is the assignment problem: given n workers and n jobs, with a cost c[i][j] for assigning worker i to job j, find the minimum total cost to assign each worker to a unique job. The state is dp[mask] = the minimum cost to assign the first popcount(mask) workers to the jobs indicated by mask. The transition: let i = popcount(mask), then dp[mask] = min(dp[mask ^ (1 << j)] + c[i][j]) over all j in mask. The base case is dp[0] = 0. The answer is dp[(1 << n) - 1]. The complexity is O(2^n * n), which is feasible for n up to about 20.

```
// Assignment problem (Hungarian for n>20, bitmask DP for n<=20)
int n; // number of workers/jobs
vector<vector<int>> c; // cost matrix

vector<int> dp(1 << n, INT_MAX);
dp[0] = 0;
for (int mask = 0; mask < (1 << n); mask++) {
    int i = __builtin_popcount(mask); // worker being assigned
    for (int j = 0; j < n; j++) {
        if (mask & (1 << j)) {
            int prev = mask ^ (1 << j);
            if (dp[prev] != INT_MAX) {
                dp[mask] = min(dp[mask], dp[prev] + c[i][j]);
            }
```

```
        }
    }
}
// Answer: dp[(1 << n) - 1]
```

Another classic is Hamiltonian path: find the shortest path that visits every node exactly once. The state is (mask, v) — the shortest path that visits the nodes in mask and ends at v. The transition: dp[mask][v] = min(dp[mask ^ (1 << v)][u] + dist[u][v]) over all u in mask with u != v. The base case is dp[1 << v][v] = 0. The answer is min(dp[(1 << n) - 1][v]) over all v. The complexity is O(2^n * n^2), feasible for n up to about 18. This is the foundation of the Travelling Salesman Problem (TSP).

When Bitmask DP Applies

The signal that bitmask DP applies is a small n (≤ 20) combined with a problem that involves selecting, permuting, or matching elements. If n is small and the problem feels like it requires trying many combinations, bitmask DP is likely the answer. The constraint n ≤ 20 in the problem statement is essentially a hint from the problemsetter that bitmask DP is intended — 2^20 is about 10^6, which fits comfortably in time and memory.

A common pitfall is using bitmask DP when a simpler approach works. For example, the assignment problem can also be solved by the Hungarian algorithm in O(n^3), which is faster for n > 20. Before committing to bitmask DP, ask whether there is a polynomial algorithm that works for the given n. If yes, use the polynomial algorithm; if no, or if n is small enough that the bitmask DP is faster, use the bitmask DP.

> *Bitmask DP applies when n ≤ 20 and the problem involves subsets, permutations, or matchings. Memorise the subset iteration trick: for (sub = (mask-1) & mask; sub > 0; sub = (sub-1) & mask).*

Chapter 15 — Digit DP

COUNTING NUMBERS IN A RANGE WITH A PROPERTY

Digit DP is a technique for counting numbers in a range [L, R] that satisfy a given property. The property might be "sum of digits equals S", "no two adjacent digits are equal", "the number is divisible by K", and so on. The naive approach — iterate over all numbers in the range and check each — is O(R - L), which is infeasible for large ranges (R up to 10^18). Digit DP solves these problems in O(log R * state) by processing the number one digit at a time. This chapter covers the standard template and the state design that makes it work.

The State

The digit DP state is (pos, tight, lead_zero, ...). The pos parameter is the current digit position (from most significant to least). The tight parameter is a boolean indicating whether the prefix built so far is exactly equal to the prefix of the upper bound — if so, the next digit is restricted to be at most the corresponding digit of the upper bound; if not, the next digit can be anything from 0 to 9. The lead_zero parameter is a boolean indicating whether the prefix built so far is all zeros — this matters because leading zeros should not be counted as actual digits in the property being checked. The "..." represents problem-specific state — for "sum of digits equals S", this would be the current sum; for "divisible by K", this would be the current number mod K.

The number of states is O(num_digits * 2 * 2 * problem_state_size). For a number up to 10^18, num_digits is at most 19, so even with a problem state size of 100, the total state space is around 19 * 4 * 100 = 7600, which is trivially small. The transition is to try each possible next digit (0-9, restricted by tight), update the state accordingly, and recurse. The complexity is O(num_digits * 10 * state_size), which is essentially O(1) for any reasonable problem.

The Standard Template

The standard digit DP implementation uses top-down memoization, because the tight and lead_zero flags make the iteration order for bottom-up awkward. The template looks like this:

```
string num; // upper bound as a string
int n; // number of digits
map<tuple<int,int,int,int>, long long> memo;

// State: (pos, tight, lead_zero, problem_state)
long long solve(int pos, bool tight, bool lead_zero, int pstate) {
    if (pos == n) {
        // Base case: check if the built number satisfies the property
        return /* 1 if satisfies, 0 else */;
    }
    auto key = make_tuple(pos, tight, lead_zero, pstate);
    if (memo.count(key)) return memo[key];

    int limit = tight ? (num[pos] - '0') : 9;
    long long result = 0;
    for (int d = 0; d <= limit; d++) {
        bool new_tight = tight && (d == limit);
        bool new_lead = lead_zero && (d == 0);
        int new_pstate = /* update pstate based on d */;
        result += solve(pos + 1, new_tight, new_lead, new_pstate);
    }
    return memo[key] = result;
}
```

The answer for the range [L, R] is solve(R) - solve(L-1), where solve(X) counts numbers in [0, X] satisfying the property. To handle L = 0, treat solve(-1) as 0. The "tight" flag ensures we never count numbers greater than X; the "lead_zero" flag ensures we handle numbers with fewer digits than X correctly (e.g., the number 5 has digit 0 in the leading positions, but those zeros should not contribute to the sum of digits).

Common Variants

The most common digit DP variant is "sum of digits equals S": count numbers in [L, R] whose digit sum is exactly S. The problem state is the current digit sum, capped at S (any sum greater than S can be treated as a single "overflow" state). The transition adds the current digit to the sum. The base case returns 1 if the sum equals S, 0 otherwise. Another common variant is "no two adjacent digits equal": the problem state is the previous digit (or a special value for "no previous digit", i.e., lead_zero). The transition checks that the current digit differs from the previous digit.

A third common variant is "divisible by K": the problem state is the current number mod K. The transition is new_pstate = (pstate * 10 + d) % K. The base case returns 1 if pstate == 0 (i.e., the number is divisible by K). The state space is num_digits * 2 * 2 * K, so this is feasible for K up to about 10^4. For larger K, the state

space becomes too large and you need a different approach (often matrix exponentiation, which is beyond this guide).

> *Digit DP counts numbers in a range with a property, in O(log R * state) instead of O(R). The state is (pos, tight, lead_zero, problem_state) — memorise this template.*

Chapter 16 — Common DP Pitfalls and Optimisation

DEBUGGING AND SPEEDING UP YOUR DP SOLUTIONS

DP solutions fail in predictable ways. This chapter covers the most common pitfalls — the bugs and design errors that cause DP solutions to produce wrong answers, time out, or crash — and the standard optimisations that take a too-slow DP and make it pass. Mastering these is the difference between a climber who can solve DP problems and a climber who can solve them reliably under contest conditions.

State Explosion

The most common DP design error is adding an unnecessary state dimension. Every additional dimension multiplies the state space by its size, so a state that should be 2D can easily become 3D and balloon from 10^6 states to 10^9. The defence is to constantly ask: "do I actually need this dimension, or can I derive it from the other dimensions?" For example, in a knapsack with items sorted by weight, you do not need a separate dimension for the current weight — it is determined by which items have been selected. In a path-counting DP, you do not need separate dimensions for row and column if the path is constrained to move only right and down — the sum of the dimensions gives the step number, so you can use a 1D state indexed by row, with the column derived from the step number.

A specific case of state explosion is the "extra dimension for the answer" anti-pattern. Sometimes a problem asks for both the maximum value and the number of ways to achieve it. The naive approach is to have two DP tables — one for the max, one for the count. The better approach is to have one DP table where each cell stores both values, because the count depends on the max and they share the same state space. Storing them separately can double the state space unnecessarily.

Wrong Base Cases

The second most common DP error is wrong base cases. The base case is the boundary of the state space — the smallest subproblem that does not depend on any other. Getting it wrong produces either off-by-one errors (the answer is shifted by 1) or undefined behaviour (the DP reads from negative indices). The defence is to explicitly write out the base case for every state dimension's boundary. For a 2D DP, the base cases are typically dp[0][j] for all j and dp[i][0] for all i — specify both explicitly. For a 1D DP, the base case is usually dp[0] or dp[1] — specify it explicitly and check that the transition never reads from a negative index.

A subtle base case error is forgetting the "empty" subproblem. In knapsack, dp[0][w] = 0 for all w (with no items, the best value is 0) — this is correct. In edit distance, dp[0][j] = j and dp[i][0] = i — this is correct because editing to/from an empty string requires inserting/deleting all characters. The general rule: the base case should reflect the natural answer for the smallest possible subproblem, even if that subproblem is trivial.

Skip this and the DP will produce wrong answers on edge cases.

Missing Transitions

A missing transition is a choice the DP does not consider, leading to a suboptimal answer. For example, in knapsack, the transition is "skip item i" or "take item i" — forgetting either one gives wrong answers. In edit distance, the transition is "insert", "delete", or "replace" — forgetting one gives wrong answers. The defence is to enumerate all possible "last decisions" explicitly before writing the transition. Ask: "what are all the ways the subproblem could end?" Each way is a transition.

A specific missing transition is the "do nothing" option. In many problems, the optimal solution for a subproblem is to take no action — skip the current element, make no edit, include no item. This is often the base case of the transition, but it is easy to forget when the problem is phrased as "maximise" or "minimise" rather than "count". Always ask: "can the optimal solution be the trivial one where nothing happens?" If yes, include the identity transition.

Dimension Reduction

A common DP optimisation is dimension reduction — collapsing a 2D DP into a 1D DP when the 2D DP only depends on the previous row. The knapsack example earlier showed this: the 2D dp[i][w] was reduced to a 1D dp[w] by iterating w backwards. This works because dp[i][w] only depends on dp[i-1][w] and dp[i-1][w - weight[i]], so once row i is computed, row i-1 is no longer needed. The backwards iteration ensures that the values being read are from row i-1, not the partially-computed row i. This optimisation reduces the memory from O(nW) to O(W), which can be the difference between MLE and AC.

Prefix-Sum Optimisation

When a DP transition sums over a range of previous states, the naive transition is O(n) per state, giving O(n^2) total. The prefix-sum optimisation precomputes prefix sums of the DP table, allowing the transition to be O(1). For example, if dp[i] = sum(dp[j]) for j from i-k to i-1, you can precompute prefix[i] = sum(dp[j]) for j from 0 to i-1, and then dp[i] = prefix[i] - prefix[i-k]. This reduces the total complexity from O(n^2) to O(n). Many DP problems at 1500-1700 difficulty require this optimisation to pass the time limit.

Monotonic Queue Optimisation

When a DP transition takes the max (or min) over a sliding window of previous states, the monotonic queue optimisation applies. The naive transition is O(n) per state; the monotonic queue gives O(1) amortised. The pattern is the same as the sliding-window max from Chapter 10: maintain a deque of indices whose dp values are decreasing, so the front of the deque is always the max. This optimisation appears in problems like "jump game with maximum score" or "partition array into k subarrays with minimum cost".

When to Switch from DP to Greedy

Sometimes a problem that looks like DP is actually solvable by a greedy algorithm. The signal is that the DP transition has a particularly simple form — usually "take the best of a small number of options" — that suggests a greedy choice might work. If you suspect greedy might apply, try to construct a counterexample. If

you cannot construct one after 2-3 minutes, the greedy is probably correct and you should use it instead of DP — greedy is faster to code and faster to run. If you can construct a counterexample, stick with DP.

> *DP fails in predictable ways: state explosion, wrong base cases, missing transitions. Learn the standard optimisations — dimension reduction, prefix sums, monotonic queues — and apply them when the naive DP is too slow.*

PART IV

Graph Algorithms Deep Dive

Graphs appear in nearly every Division 2 round from 1300 upward. This part covers representation and traversal, shortest paths, MST and DSU, topological sort, SCC and 2-SAT, and network flow — the complete toolkit needed to handle any graph problem through 1800 difficulty.

Chapter 17 — Graph Representation and Traversal

THE FOUNDATION EVERY GRAPH ALGORITHM BUILDS ON

Graph traversal — BFS and DFS — is the foundation of every other graph algorithm. Shortest paths, MST, topological sort, SCC, and network flow all build on traversal. At the 1300-1600 band, traversal itself appears in many problems: connectivity, component counting, cycle detection, bipartiteness checking, and flood-fill on grids all reduce to BFS or DFS. This chapter covers the standard representations, the two traversal algorithms, and the common bugs that cause them to fail.

Graph Representation

There are three standard representations: adjacency list, adjacency matrix, and edge list. The adjacency list (vector> in C++) is the default — it uses O(V + E) memory and supports O(deg(v)) iteration over neighbours of v, which is what most graph algorithms need. Use the adjacency list unless you have a specific reason not to. The adjacency matrix (vector> with V rows and V columns) uses O(V^2) memory and supports O(1) edge existence queries, but iteration over neighbours is O(V). Use the matrix only when V is small (≤ 500) and you need fast edge existence checks, as in Floyd-Warshall. The edge list (vector> or vector>) is used in MST algorithms (Kruskal) and in problems where you need to sort edges.

```
// Adjacency list construction (undirected)
int n, m; // n vertices, m edges
cin >> n >> m;
vector<vector<int>> g(n);
for (int i = 0; i < m; i++) {
    int u, v; cin >> u >> v;
    u--; v--; // convert to 0-indexed if input is 1-indexed
    g[u].push_back(v);
    g[v].push_back(u);
}

// Weighted adjacency list
vector<vector<pair<int,long long>>> gw(n);
// gw[u].push_back({v, weight});
```

BFS — Breadth-First Search

BFS explores vertices in order of distance from a source, using a queue. It computes shortest paths in unweighted graphs (or graphs with unit-weight edges). The standard implementation uses a queue and a visited/dist array. BFS is O(V + E) and is the right choice for: shortest paths on unweighted graphs, level-order traversal, bipartiteness checking (via 2-colouring), and connectivity in unweighted graphs.

```
// BFS shortest path on unweighted graph
vector<int> bfs(vector<vector<int>>& g, int src) {
    int n = g.size();
    vector<int> dist(n, -1);
    queue<int> q;
    dist[src] = 0;
    q.push(src);
    while (!q.empty()) {
```

```
        int v = q.front(); q.pop();
        for (int u : g[v]) {
            if (dist[u] == -1) {
                dist[u] = dist[v] + 1;
                q.push(u);
            }
        }
    }
    return dist;
}
```

DFS — Depth-First Search

DFS explores as far as possible along each branch before backtracking, using a stack (either the call stack via recursion or an explicit stack). DFS is O(V + E) and is the right choice for: cycle detection, topological sort, SCC, connectivity in directed graphs, and any problem involving the structure of paths (not just their length). The recursive implementation is cleaner but can stack overflow on deep graphs; the iterative implementation is safer but slightly more verbose.

```
// Recursive DFS with visited array
vector<vector<int>> g;
vector<bool> visited;

void dfs(int v) {
    visited[v] = true;
    for (int u : g[v]) {
        if (!visited[u]) dfs(u);
    }
}

// Iterative DFS (avoids stack overflow)
void dfs_iter(int start) {
    vector<bool> vis(g.size(), false);
    stack<int> st;
    st.push(start);
    while (!st.empty()) {
        int v = st.top(); st.pop();
        if (vis[v]) continue;
        vis[v] = true;
        for (int u : g[v]) if (!vis[u]) st.push(u);
    }
}
```

When BFS vs DFS

BFS finds shortest paths in unweighted graphs; DFS does not. DFS uses less memory on shallow graphs (only the current path), while BFS uses more (the entire frontier). For connectivity (is there a path from u to v?), either works. For shortest path, use BFS. For topological sort, use DFS. For cycle detection in directed graphs, use DFS with a "in current path" flag. For bipartiteness, either works — BFS is more intuitive (alternate colours), DFS is shorter to code recursively. As a rule of thumb: if you need distances, use BFS; if you need structure (cycles, topological order, SCC), use DFS.

Common Traversal Bugs

The most common bug is visiting a vertex twice. The fix is the visited array — mark a vertex as visited when you first see it, not when you process it. In BFS, this means marking visited when you push to the queue, not when you pop. In recursive DFS, mark visited at the start of the function. The second common bug is forgetting to handle disconnected components — if the graph might be disconnected, you need to start a new traversal from each unvisited vertex. The third bug is recursion depth — on a chain of 10^5 vertices, recursive DFS will stack overflow. Use iterative DFS or increase the stack size (Codeforces allows this via compiler flags, but iterative is safer).

> *Mark visited when you first see a vertex, not when you process it. Handle disconnected components by looping over all vertices. Use iterative DFS for graphs deeper than 10^5.*

Chapter 18 — Shortest Path Algorithms

DIJKSTRA, BELLMAN-FORD, FLOYD-WARSHAR, 0-1 BFS

Shortest path problems ask for the minimum-weight path between vertices. The algorithm choice depends on the graph structure: Dijkstra for non-negative weights, Bellman-Ford for graphs with negative weights, Floyd-Warshall for all-pairs shortest paths, and 0-1 BFS for graphs with only 0 and 1 weight edges. Each algorithm has a specific use case and a specific complexity, and choosing the wrong one is a common cause of TLE or wrong answers. This chapter covers the four algorithms with code templates and complexity analysis.

Dijkstra's Algorithm

Dijkstra computes shortest paths from a single source to all other vertices in a graph with non-negative edge weights. The algorithm maintains a priority queue of (distance, vertex) pairs, extracts the minimum, and relaxes edges out of it. The complexity is O((V + E) log V) with a binary heap. The standard implementation uses std::priority_queue with lazy deletion (skip a vertex if its distance has already been improved).

```
// Dijkstra with priority_queue and lazy deletion
vector<long long> dijkstra(vector<vector<pair<int,long long>>>& g, int src) {
    int n = g.size();
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, v] = pq.top(); pq.pop();
        if (d > dist[v]) continue; // lazy deletion
        for (auto [u, w] : g[v]) {
            if (dist[v] + w < dist[u]) {
                dist[u] = dist[v] + w;
                pq.push({dist[u], u});
            }
        }
    }
    return dist;
}
```

A common bug is forgetting the lazy deletion check (if (d > dist[v]) continue). Without it, the algorithm still produces correct distances but processes each vertex multiple times, degrading the complexity to O(V * E) in the worst case. Always include the lazy deletion check. Another common bug is using Dijkstra on a graph with negative edge weights — Dijkstra produces wrong answers in this case; use Bellman-Ford instead.

Bellman-Ford Algorithm

Bellman-Ford computes shortest paths from a single source in a graph that may have negative edge weights. It also detects negative cycles reachable from the source. The algorithm relaxes all edges V-1 times; if a further relaxation is possible, there is a negative cycle. The complexity is O(V * E), which is slower than Dijkstra but handles negative weights. Use Bellman-Ford when the graph has negative weights or when you need to detect negative cycles.

```
// Bellman-Ford: returns false if negative cycle reachable from src
struct Edge { int u, v; long long w; };

bool bellman_ford(int n, vector<Edge>& edges, int src, vector<long long>& dist) {
    dist.assign(n, LLONG_MAX);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) {
        for (auto& e : edges) {
            if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
            }
        }
    }
    // Check for negative cycle
    for (auto& e : edges) {
        if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v]) {
            return false; // negative cycle detected
        }
    }
    return true;
}
```

Floyd-Warshall Algorithm

Floyd-Warshall computes all-pairs shortest paths in O(V^3). It works with negative edge weights (but not negative cycles). The algorithm is a triple nested loop over an intermediate vertex k, with the inner loop updating dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]). Use Floyd-Warshall when V is small (≤ 500) and you need distances between all pairs, or when you need to detect negative cycles (check if any dist[i][i] < 0 after the algorithm).

```
// Floyd-Warshall: all-pairs shortest paths, O(V^3)
void floyd_warshall(vector<vector<long long>>& dist) {
    int n = dist.size();
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != LLONG_MAX && dist[k][j] != LLONG_MAX) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
```

```
        }
    }
}
```

0-1 BFS

When all edge weights are 0 or 1, you can compute shortest paths in O(V + E) using a deque instead of a priority queue. The trick: when relaxing an edge of weight 0, push the vertex to the front of the deque; when relaxing an edge of weight 1, push to the back. This ensures the deque stays sorted by distance, giving the same effect as Dijkstra but faster. Use 0-1 BFS whenever the problem has only 0/1 edge weights — it is significantly faster than Dijkstra.

```
// 0-1 BFS: shortest paths with 0/1 edge weights
vector<int> bfs01(vector<vector<pair<int,int>>>& g, int src) {
    int n = g.size();
    vector<int> dist(n, INT_MAX);
    deque<int> dq;
    dist[src] = 0;
    dq.push_front(src);
    while (!dq.empty()) {
        int v = dq.front(); dq.pop_front();
        for (auto [u, w] : g[v]) {
            if (dist[v] + w < dist[u]) {
                dist[u] = dist[v] + w;
                if (w == 0) dq.push_front(u);
                else dq.push_back(u);
            }
        }
    }
    return dist;
}
```

Dijkstra for non-negative weights, Bellman-Ford for negative weights or cycle detection, Floyd-Warshall for all-pairs (small V), 0-1 BFS for 0/1 weights. Choose based on graph structure — the wrong choice means TLE or wrong answers.

Chapter 19 — Minimum Spanning Tree and Disjoint Set Union

KRUSKAL, PRIM, AND THE DSU DATA STRUCTURE

A minimum spanning tree (MST) of a weighted undirected graph is a subset of edges that connects all vertices with minimum total weight and contains no cycles. MST problems appear frequently at 1400-1700 difficulty, often disguised as "connect the cities with minimum road length" or "wire the components with minimum cost". The two standard algorithms are Kruskal (sort edges by weight, add edges that connect different components) and Prim (grow a tree from a starting vertex by adding the cheapest edge to a new vertex). Both rely on the disjoint set union (DSU) data structure for efficiency.

Disjoint Set Union (DSU)

DSU (also called Union-Find) maintains a collection of disjoint sets and supports two operations: find(x) returns the representative of the set containing x, and union(x, y) merges the sets containing x and y. With path compression (in find, point each node directly to the root) and union by rank (always attach the smaller tree to the larger), each operation is amortised O(alpha(n)), which is essentially O(1). DSU is one of the most useful data structures in competitive programming — learn it cold.

```
// DSU with path compression and union by rank
struct DSU {
    vector<int> parent, rank_;
    DSU(int n) : parent(n), rank_(n, 0) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    bool unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false; // already in same set
        if (rank_[px] < rank_[py]) swap(px, py);
        parent[py] = px;
        if (rank_[px] == rank_[py]) rank_[px]++;
        return true;
    }
    bool same(int x, int y) { return find(x) == find(y); }
};
```

Kruskal's Algorithm

Kruskal sorts all edges by weight, then iterates through them, adding each edge to the MST if it connects two different components (checked via DSU). The result is the MST. The complexity is O(E log E) for the sort, which dominates. Kruskal is the standard MST algorithm in competitive programming because it is simple to implement and works with the edge list representation that is natural for most problems.

```
// Kruskal MST
struct Edge { int u, v; long long w; };

long long kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return a.w < b.w;
    });
    DSU dsu(n);
    long long total = 0;
    int count = 0;
    for (auto& e : edges) {
        if (dsu.unite(e.u, e.v)) {
            total += e.w;
            count++;
            if (count == n - 1) break; // MST complete
        }
    }
    return total; // -1 if graph not connected (count < n-1)
}
```

Prim's Algorithm

Prim grows the MST from a starting vertex by repeatedly adding the cheapest edge connecting a tree vertex to a non-tree vertex. It uses a priority queue of edges. The complexity is O(E log V), similar to Dijkstra. Prim is useful when you already have an adjacency list representation and do not want to build an edge list, but Kruskal is usually simpler. Use Prim when the graph is dense (E close to V^2), where sorting edges in Kruskal becomes expensive.

DSU Applications Beyond MST

DSU has many applications beyond MST: connectivity queries (are u and v in the same component?), cycle detection in undirected graphs (an edge connecting two already-connected vertices forms a cycle), offline range queries (with the small-to-large merge trick), and DSU with rollback (for problems where you need to undo unions, such as dynamic connectivity on a line). The basic DSU above covers 90% of contest problems; the rollback variant is needed for the other 10% and is a topic worth studying separately.

> *DSU with path compression and union by rank is amortised O(alpha(n)) per operation — essentially O(1). Learn it cold; it appears in MST, connectivity, cycle detection, and more.*

Chapter 20 — Topological Sort and DAGs

ORDERING VERTICES WITH DEPENDENCIES

A topological sort of a directed acyclic graph (DAG) is a linear ordering of vertices such that for every directed edge u -> v, u comes before v in the ordering. Topological sort is used in scheduling problems (where edges represent dependencies), in DP on DAGs (where the topological order gives the correct DP iteration order), and in cycle detection (if no topological order exists, the graph has a cycle). This chapter covers the two standard algorithms — Kahn's (BFS-based) and DFS-based — and the DP-on-DAG pattern.

Kahn's Algorithm (BFS-based)

Kahn's algorithm computes a topological order by repeatedly removing vertices with no incoming edges (in-degree 0). The algorithm: compute in-degree of all vertices, push all in-degree-0 vertices to a queue, then repeatedly pop a vertex, add it to the order, and decrement the in-degree of its neighbours (pushing any that reach in-degree 0). If the final order contains all vertices, the graph is a DAG; if not, it has a cycle. The complexity is O(V + E).

```
// Kahn's topological sort
vector<int> topo_sort(int n, vector<vector<int>>& g) {
    vector<int> in_degree(n, 0);
    for (int u = 0; u < n; u++) for (int v : g[u]) in_degree[v]++;
    queue<int> q;
    for (int i = 0; i < n; i++) if (in_degree[i] == 0) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int v = q.front(); q.pop();
        order.push_back(v);
        for (int u : g[v]) {
            if (--in_degree[u] == 0) q.push(u);
        }
    }
    // If order.size() < n, graph has a cycle
```

```
    return order;
}
```

DFS-based Topological Sort

The DFS-based algorithm produces a topological order by appending vertices to the order in reverse finish time of a DFS. The algorithm: do a DFS from each unvisited vertex, and when you finish processing a vertex (after all its neighbours are visited), prepend it to the order. The result is a valid topological order. This algorithm is shorter to code than Kahn's but does not naturally detect cycles — you need an additional "in current path" flag to detect cycles during the DFS.

DP on DAG

Many problems on DAGs reduce to DP, where the topological order gives the correct iteration order. The classic examples: longest path in a DAG (process vertices in topological order, dp[v] = max(dp[u] + weight(u, v)) over all incoming edges), count of paths from s to t (dp[v] = sum(dp[u]) over all incoming edges, with dp[s] = 1), and reachability (transitive closure via DP). The topological order is essential because it guarantees that when you process vertex v, all vertices that can reach v have already been processed.

```
// Longest path in DAG (unweighted, in vertices)
vector<int> longest_path(int n, vector<vector<int>>& g, int src) {
    vector<int> order = topo_sort(n, g);
    vector<int> dp(n, INT_MIN);
    dp[src] = 0;
    for (int v : order) {
        if (dp[v] == INT_MIN) continue;
        for (int u : g[v]) {
            dp[u] = max(dp[u], dp[v] + 1);
        }
    }
    return dp;
}
```

Cycle Detection in Directed Graphs

A directed graph has a topological order iff it is a DAG (no cycles). So one way to detect a cycle is to run Kahn's algorithm and check if the order contains all vertices. Another way is DFS with three states: unvisited, in-current-path (grey), and finished (black). If DFS encounters a grey vertex, there is a cycle. The DFS approach is more useful when you also want to extract the cycle (by tracing back through the path).

> *Topological sort gives the iteration order for DP on DAGs. Use Kahn's for simplicity and cycle detection, DFS-based when you need to extract the cycle path.*

Chapter 21 — Strongly Connected Components and 2-SAT

COMPRESSING DIRECTED GRAPHS AND SOLVING SATISFIABILITY

A strongly connected component (SCC) of a directed graph is a maximal subset of vertices such that every vertex is reachable from every other vertex in the subset. Compressing a graph by its SCCs produces a DAG (the condensation), which is much easier to work with. SCC is the foundation of 2-SAT, a constraint satisfaction problem that appears in many 1500-1800 contests. This chapter covers the two standard SCC algorithms (Tarjan and Kosaraju) and the 2-SAT reduction.

Tarjan's SCC Algorithm

Tarjan's algorithm finds all SCCs in a single DFS, using the low-link technique. For each vertex, maintain a "discovery time" (when it was first visited) and a "low-link" (the smallest discovery time reachable from the vertex via tree edges plus at most one back edge or cross edge). When you finish processing a vertex and its low-link equals its discovery time, it is the root of an SCC — pop vertices from a stack until you reach it. The complexity is O(V + E). Tarjan is the standard SCC algorithm in competitive programming because it is a single DFS and produces SCCs in reverse topological order of the condensation.

Kosaraju's Algorithm

Kosaraju's algorithm finds SCCs in two passes: first, do a DFS on the original graph and push vertices to a stack in finish-time order; second, do a DFS on the reversed graph, popping vertices from the stack and starting a new DFS for each unvisited vertex — each DFS tree is an SCC. Kosaraju is conceptually simpler than Tarjan but requires two passes and building the reversed graph. Both algorithms are O(V + E); choose based on which you find easier to implement correctly under contest pressure.

Condensation DAG

Once you have the SCCs, you can build the condensation DAG: each SCC becomes a single vertex, and there is an edge from SCC A to SCC B if there is an edge from any vertex in A to any vertex in B in the original graph. The condensation is always a DAG (because if there were a cycle in the condensation, the corresponding SCCs would actually be a single larger SCC). Many problems become easier on the condensation — for example, "count the number of paths from s to t" is intractable on a general directed graph but becomes a DP on the condensation DAG.

2-SAT

2-SAT is the problem of determining whether a boolean formula in conjunctive normal form (CNF) with exactly 2 literals per clause is satisfiable. The key insight is that each clause (a OR b) can be rewritten as two implications: (NOT a -> b) and (NOT b -> a). Building the implication graph (vertices are literals, edges are implications) and finding SCCs gives the answer: the formula is satisfiable iff no literal and its negation are in the same SCC. To extract a satisfying assignment, process SCCs in reverse topological order; for each literal, if it has not been assigned, assign it true if its SCC comes after its negation's SCC in topological order, false otherwise.

2-SAT problems appear in many disguises: "can you assign values to variables such that each pair satisfies a constraint?" is the canonical form. Common variants: choosing one of two options per variable with pairwise exclusions, scheduling with conflict constraints, and graph colouring with two colours under additional constraints. The recognition skill is spotting that the constraints can be written as 2-literal clauses; once you see

that, the 2-SAT machinery applies mechanically.

SCC compresses a directed graph into a DAG. 2-SAT reduces a satisfiability problem to SCC on the implication graph. Both are standard tools for 1500-1800 graph problems.

Chapter 22 — Network Flow Basics

MAX-FLOW, MIN-CUT, AND BIPARTITE MATCHING

Network flow is a family of problems involving the movement of "flow" through a graph from a source to a sink, subject to capacity constraints on edges. The most basic problem is max-flow: given a graph with edge capacities, find the maximum amount of flow that can be sent from source to sink. Max-flow is the foundation of bipartite matching, min-cut, and many other problems. At the Expert level (1500-1600), full flow problems are rare, but bipartite matching appears often enough that you should know the reduction. This chapter covers the basics of max-flow and bipartite matching.

Max-Flow Min-Cut Theorem

The max-flow min-cut theorem states that the maximum flow from source to sink equals the minimum capacity of a cut (a partition of vertices into two sets, one containing the source and one containing the sink, where the cut capacity is the sum of capacities of edges going from the source side to the sink side). This theorem connects two seemingly different problems — flow optimisation and cut minimisation — and is the foundation of many flow applications. For example, "minimum number of edges to remove to disconnect s from t" is a min-cut problem, solvable by max-flow with unit capacities.

Ford-Fulkerson and Edmonds-Karp

Ford-Fulkerson is the basic max-flow algorithm: repeatedly find an augmenting path (a path from source to sink in the residual graph) and push as much flow as possible along it. The algorithm terminates when no augmenting path exists. With BFS for finding augmenting paths, it becomes Edmonds-Karp, which runs in O(V * E^2). For most contest problems at this level, Edmonds-Karp is fast enough. For problems with large graphs, Dinic's algorithm (which uses BFS to build level graphs and DFS to find blocking flows) is faster, at O(V^2 * E).

Bipartite Matching

A bipartite graph is one whose vertices can be partitioned into two sets such that all edges go between the sets. A matching is a set of edges with no shared vertices. The maximum bipartite matching problem — find the largest matching in a bipartite graph — appears in many contest problems: assigning tasks to workers, pairing elements with constraints, and various "maximum independent set on bipartite graphs" problems (which equals V minus maximum matching by Konig's theorem). The reduction to max-flow: add a super-source connected to all left vertices, a super-sink connected from all right vertices, direct all original edges from left to right, give every edge capacity 1, and compute max-flow from super-source to super-sink.

```
// Bipartite matching via max-flow (sketch)
// Left vertices: 0..n-1, Right vertices: 0..m-1
// Source = n+m, Sink = n+m+1
```

```
// Edges: source->left (cap 1), left->right (cap 1, for each graph edge),
// right->sink (cap 1)
// Max matching = Max flow from source to sink.
```

For bipartite matching specifically, the Hungarian (Kuhn) algorithm is a simpler alternative that runs in O(V * E) and is easier to code than a full max-flow. The algorithm: for each left vertex, try to find an augmenting path to an unmatched right vertex via DFS; if found, augment the matching along the path. The complexity is O(V * E) because each augmenting path DFS is O(E) and there are at most V augmentations. For most contest problems, Kuhn is fast enough and is the recommended implementation.

When Flow Appears in Problems

Flow problems rarely appear as "compute the max-flow of this graph". Instead, they appear as word problems that can be modelled as flow. The signal to look for: a problem involving "maximum matching", "minimum cut", "assigning items with constraints", or "maximum number of disjoint paths". When you see one of these, consider whether a flow reduction applies. At the 1500-1600 level, bipartite matching is the most common flow application; full max-flow problems are more common at 1800+.

> *Max-flow min-cut connects flow optimisation to cut minimisation. Bipartite matching reduces to max-flow with unit capacities. At Expert level, Kuhn's algorithm for bipartite matching is the flow technique you will use most.*

PART V

Math and Number Theory Essentials

The math toolkit needed for Expert level: modular arithmetic, the modular inverse, sieve and prime factorisation, GCD and LCM, and basic combinatorics. These show up in nearly every contest and are the foundation of more advanced techniques like matrix exponentiation.

Chapter 23 — Modular Arithmetic and Number Theory

THE MATH YOU CANNOT AVOID IN COMPETITIVE PROGRAMMING

Modular arithmetic is the single most common mathematical technique in competitive programming. A huge fraction of problems ask for the answer "modulo 10^9 + 7" or "modulo 998244353", because the actual answer is too large to print and the modular answer is sufficient to verify correctness. The key fact is that addition, subtraction, and multiplication commute with the modulo operation — you can take mod at every step without changing the final answer. Division, however, is trickier: you cannot simply divide and then take mod, because the division might not be defined in modular arithmetic. This chapter covers the standard techniques for working with modular arithmetic and the number theory that supports them.

The Mod Everywhere Discipline

When a problem asks for the answer modulo p, you must take mod at every step of the computation, not just at the end. The reason is integer overflow: the actual answer might be a number with thousands of digits, which does not fit in any standard integer type. By taking mod at every step, you keep all intermediate values smaller than p, which fits in a long long. The three rules: (a + b) mod p = ((a mod p) + (b mod p)) mod p, (a - b) mod p = ((a mod p) - (b mod p) + p) mod p (the +p handles negative results), and (a * b) mod p = ((a mod p) * (b mod p)) mod p. Apply these rules religiously; missing a single mod can cause an overflow that produces wrong answers.

Modular Inverse

Division in modular arithmetic is multiplication by the modular inverse. The modular inverse of a modulo p is a number b such that (a * b) mod p = 1. The inverse exists iff gcd(a, p) = 1. When p is prime (as 10^9 + 7 is), every non-zero a has an inverse, given by a^(p-2) mod p by Fermat's little theorem. Compute the inverse using fast exponentiation in O(log p) time. For non-prime p, use the extended Euclidean algorithm to find the inverse, which works iff gcd(a, p) = 1.

```
// Modular arithmetic with prime mod (e.g., 1e9+7)
const long long MOD = 1e9 + 7;

long long power(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) result = result * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return result;
}

// Modular inverse for prime mod: a^(p-2) mod p
long long mod_inverse(long long a, long long p) {
    return power(a, p - 2, p);
}
```

```
// Division: a / b mod p = a * inverse(b) mod p
long long mod_div(long long a, long long b, long long p) {
    return a % p * mod_inverse(b, p) % p;
}
```

Sieve of Eratosthenes

The sieve of Eratosthenes generates all primes up to n in O(n log log n) time. The standard implementation uses a boolean array; mark each prime, then mark all its multiples as composite. For most contest problems, the standard sieve is fast enough up to n = 10^7. For larger n, use the linear sieve, which runs in O(n) but is more complex. The sieve also lets you compute the smallest prime factor of each number, which is useful for fast prime factorisation.

```
// Sieve of Eratosthenes
vector<bool> sieve(int n) {
    vector<bool> is_prime(n + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; (long long)i * i <= n; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i) {
                is_prime[j] = false;
            }
        }
    }
    return is_prime;
}
```

GCD and LCM

The greatest common divisor (GCD) of two numbers is the largest number that divides both. The least common multiple (LCM) is the smallest number that both divide. The relationship is gcd(a, b) * lcm(a, b) = a * b, so lcm(a, b) = a / gcd(a, b) * b (divide first to avoid overflow). C++ provides std::gcd and std::lcm in since C++17. The Euclidean algorithm for GCD is: gcd(a, b) = gcd(b, a mod b), with base case gcd(a, 0) = a. This runs in O(log min(a, b)) time. GCD is the foundation of much of number theory and appears in many contest problems involving fractions, divisibility, or modular arithmetic.

Prime Factorisation

Prime factorisation of n is the decomposition of n into a product of primes. The naive approach — trial division up to sqrt(n) — is O(sqrt(n)) per query, which works for n up to about 10^12. For multiple queries, precompute the smallest prime factor (SPF) of each number up to n using a modified sieve, then factorise by repeatedly dividing by the SPF. This is O(log n) per query after O(n log log n) precomputation. Prime factorisation appears in problems involving divisor counting, Euler's totient, and various number-theoretic identities.

> *Take mod at every step. Division is multiplication by the modular inverse. Precompute smallest prime factors with a sieve for fast factorisation. These three habits cover most math problems at this level.*

Chapter 24 — Combinatorics Basics

COUNTING WITH MODULAR ARITHMETIC

Combinatorics problems ask you to count things: number of ways to choose, number of ways to arrange, number of valid configurations. At the 1300-1600 band, combinatorics problems usually reduce to computing nCr (binomial coefficient) or nPr (permutation) modulo a prime, often combined with a simple DP. This chapter covers the standard techniques for computing binomial coefficients efficiently and the common combinatorial identities you need to know.

Binomial Coefficients Modulo p

The binomial coefficient nCr (n choose r) is the number of ways to choose r items from n. The direct formula is n! / (r! * (n-r)!), but computing factorials and dividing is infeasible in modular arithmetic because division requires the modular inverse. The standard approach: precompute factorials and inverse factorials up to n, then nCr = fact[n] * inv_fact[r] * inv_fact[n-r] mod p. Precomputation is O(n), each query is O(1). This is the only way to compute large binomial coefficients efficiently.

```
// Precompute factorials and inverse factorials for nCr mod p
const int MAXN = 2e5;
const long long MOD = 1e9 + 7;
long long fact[MAXN + 1], inv_fact[MAXN + 1];

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; i++) fact[i] = fact[i-1] * i % MOD;
    inv_fact[MAXN] = power(fact[MAXN], MOD - 2, MOD);
    for (int i = MAXN - 1; i >= 0; i--) inv_fact[i] = inv_fact[i+1] * (i+1) % MOD;
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * inv_fact[r] % MOD * inv_fact[n - r] % MOD;
}
```

A key implementation detail: precompute the inverse factorials in reverse, using the identity inv_fact[i] = inv_fact[i+1] * (i+1) mod p. This avoids computing a separate modular inverse for each factorial, which would be O(n log p) instead of O(n). The single inverse computation is for inv_fact[MAXN], and the rest follow by multiplication.

Pascal's Identity and Recurrence

Pascal's identity states that nCr = (n-1)C(r-1) + (n-1)Cr. This is the recurrence behind Pascal's triangle and is useful for computing binomial coefficients without factorials, especially when n and r are small (≤ 5000) and you need all values. The DP is O(n^2) time and space, which is fine for small n but not for n up to 10^5. For large n, use the factorial precomputation above.

Stars and Bars

Stars and bars is a combinatorial technique for counting the number of ways to distribute n identical items into k distinct bins. The answer is (n + k - 1) choose (k - 1). The intuition: think of the n items as stars and the k-1 dividers between bins as bars; the total number of objects is n + k - 1, and you choose which k - 1 of them are bars. Stars and bars appears in many counting problems — distributing resources, counting solutions to x1 + x2

+ ... + xk = n, and various "integer partition" problems.

Inclusion-Exclusion

Inclusion-exclusion is a technique for counting the size of a union of sets when the individual set sizes and intersections are known. The formula for three sets: |A ∪ B ∪ C| = |A| + |B| + |C| - |A ∩ B| - |A ∩ C| - |B ∩ C| + |A ∩ B ∩ C|. The general pattern alternates between adding and subtracting intersections of increasing size. Inclusion-exclusion appears in problems like "count numbers up to N divisible by at least one of a1, a2, ..., ak" — the answer is the union of "divisible by ai" sets, computed via inclusion-exclusion over the LCMs of subsets.

Precompute factorials and inverse factorials for O(1) nCr queries. Stars and bars for distributions. Inclusion-exclusion for unions. These three techniques cover most combinatorics problems at this level.

PART VI

The 6-Month Practice Plan

A week-by-week themed plan calibrated for 10-15 hours per week. Each month has a clear focus, each week a specific topic, with target problem ratings and contest cadence. Adjust based on your actual progress, not on where the plan says you should be.

Chapter 25 — Month 1: Foundation and Speed (Weeks 1-4)

SOLIDIFY THE FUNDAMENTALS AND BUILD CONTEST SPEED

Month 1 has two goals: solidify the fundamental techniques (greedy, binary search, two pointers, basic math) and build contest speed on A and B problems. By the end of the month, you should be solving A and B in under 25 minutes combined in virtual contests, and you should have a clean implementation of greedy, binary search, and two-pointer templates that you can apply without thinking. The month is light on DP and graphs — those come in months 2 and 3 — because the fundamentals need to be solid before adding complex topics.

Week 1: Greedy and Sorting

Theme: greedy algorithms and sorting-based problem solving. Solve 8-10 problems at rating 1200-1400, tagged "greedy" and "sortings". Focus on recognising the sort criterion and constructing counterexamples before submitting. Run one virtual contest at Div 2 difficulty. Daily time: 1.5-2 hours. By end of week, you should be able to identify the sort criterion for a greedy problem within 5 minutes of reading the statement.

Week 2: Binary Search and Two Pointers

Theme: binary search (including binary search on the answer) and two-pointer techniques. Solve 8-10 problems at rating 1300-1500, tagged "binary search" and "two pointers". Pay special attention to the "binary search the answer" pattern — it is the highest-leverage pattern at this level. Run one virtual contest. Daily time: 1.5-2 hours. By end of week, you should be able to write the binary-search-on-answer template from memory without the off-by-one bug.

Week 3: Math and Number Theory

Theme: modular arithmetic, GCD/LCM, sieve, basic combinatorics. Solve 8-10 problems at rating 1200-1500, tagged "math" and "number theory". Implement the modular arithmetic template (fact, inv_fact, nCr) and use it in at least 3 problems. Run one virtual contest. Daily time: 1.5-2 hours. By end of week, you should have a working modular arithmetic template saved and tested.

Week 4: STL and Implementation

Theme: STL mastery and clean implementation. Solve 8-10 problems at rating 1300-1500 that require heavy STL use (sets, maps, priority queues, sorting, lower_bound). Focus on speed and correctness — these problems should be solvable in under 15 minutes each. Run two virtual contests. Daily time: 1.5-2 hours. By end of week, your A and B solve times in virtual contests should be under 25 minutes combined.

Week	Theme	Problem Rating	Count	Contests
1	Greedy + sorting	1200-1400	8-10	1 virtual
2	Binary search + two pointers	1300-1500	8-10	1 virtual
3	Math + number theory	1200-1500	8-10	1 virtual

Week	Theme	Problem Rating	Count	Contests
4	STL + implementation	1300-1500	8-10	2 virtual

Table 25.1 — Month 1 weekly breakdown.

> *Month 1 is about speed and fundamentals. If your A+B time is over 30 minutes by end of week 4, repeat week 4 before moving to month 2 — speed on the easy problems is the foundation everything else builds on.*

Chapter 26 — Month 2: DP Mastery (Weeks 5-8)

TURN DP FROM WEAKNESS TO STRENGTH

Month 2 is dedicated to dynamic programming — your weak area, and the highest-leverage topic for climbing to Expert. By the end of the month, you should be solving 1500-rated DP problems within 30 minutes and recognising DP patterns in contest statements. The month builds from 1D DP through knapsack and LIS to tree DP and bitmask DP, with each week layered on the previous. Do not skip ahead — the patterns build on each other.

Week 5: DP Fundamentals and 1D DP

Theme: the four-step DP method and 1D patterns (Fibonacci-style, house robber, climbing stairs). Solve 10-12 problems at rating 1300-1500, tagged "dp". Focus on the four-step method — define state, write transition, base case, identify answer — for every problem. Run one virtual contest focused on DP problems. Daily time: 2 hours. By end of week, the four-step method should be automatic.

Week 6: Knapsack and LIS/LCS

Theme: 2D DP patterns — 0/1 knapsack, longest increasing subsequence, longest common subsequence. Solve 10-12 problems at rating 1400-1600. Implement both the O(n^2) and O(n log n) versions of LIS. Run one virtual contest. Daily time: 2 hours. By end of week, knapsack and LIS should be templates you can apply without thinking.

Week 7: DP on Trees and Bitmask DP

Theme: tree DP (diameter, independent set, rerooting) and bitmask DP (assignment, TSP, subset enumeration). Solve 10-12 problems at rating 1500-1700. These are harder — expect to spend more time per problem and to need the editorial more often. Run one virtual contest. Daily time: 2-2.5 hours. By end of week, you should have working templates for tree DP and bitmask DP saved.

Week 8: Digit DP and DP Optimisation

Theme: digit DP (counting in ranges) and DP optimisations (prefix sum, monotonic queue, dimension reduction). Solve 10-12 problems at rating 1500-1700. The digit DP template is complex — spend extra time on it. Run one virtual contest focused on DP. Daily time: 2-2.5 hours. By end of month, your DP solving rate on 1500-rated problems should be over 70%.

Week	Theme	Problem Rating	Count	Contests
5	DP fundamentals + 1D DP	1300-1500	10-12	1 virtual
6	Knapsack + LIS/LCS	1400-1600	10-12	1 virtual
7	Tree DP + bitmask DP	1500-1700	10-12	1 virtual
8	Digit DP + optimisation	1500-1700	10-12	1 virtual

Table 26.1 — Month 2 weekly breakdown.

> *DP is learned by doing, not by reading. If you cannot solve a problem in 30 minutes, read the editorial, implement the solution, and re-attempt the problem the next day from scratch. This re-attempt step is what cements the pattern.*

Chapter 27 — Month 3: Graph Mastery (Weeks 9-12)

GRAPH FLUENCY FROM BFS TO FLOW

Month 3 tackles your other weak area: graphs. By the end of the month, you should be comfortable with BFS, DFS, shortest paths, MST, DSU, topological sort, and basic SCC. The month builds from traversal through shortest paths to more advanced techniques, with each week layering on the previous. As with month 2, do not skip ahead — the algorithms build on each other.

Week 9: BFS/DFS and Applications

Theme: graph traversal and its applications — connectivity, components, bipartiteness, cycle detection, flood fill. Solve 10-12 problems at rating 1300-1500, tagged "dfs and similar" and "bfs". Run one virtual contest. Daily time: 2 hours. By end of week, BFS and DFS should be templates you write without reference.

Week 10: Shortest Paths

Theme: Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS. Solve 10-12 problems at rating 1400-1600. Implement each algorithm and know when to use which. Run one virtual contest. Daily time: 2 hours. By end of week, you should have a tested Dijkstra template and know the use cases for the other algorithms.

Week 11: MST, DSU, and Topological Sort

Theme: Kruskal, Prim, DSU, topological sort, DP on DAG. Solve 10-12 problems at rating 1400-1700. The DSU template is essential — make sure you have it cold. Run one virtual contest. Daily time: 2 hours. By end of week, DSU and topological sort should be automatic.

Week 12: SCC and Flow Basics

Theme: Tarjan/Kosaraju SCC, condensation DAG, 2-SAT, bipartite matching. Solve 10-12 problems at rating 1500-1700. These are the hardest graph topics at this level — expect to need the editorial more often. Run one

virtual contest focused on graphs. Daily time: 2-2.5 hours. By end of month, your graph solving rate on 1500-rated problems should be over 65%.

Week	Theme	Problem Rating	Count	Contests
9	BFS/DFS + applications	1300-1500	10-12	1 virtual
10	Shortest paths	1400-1600	10-12	1 virtual
11	MST + DSU + topo sort	1400-1700	10-12	1 virtual
12	SCC + flow basics	1500-1700	10-12	1 virtual

Table 27.1 — Month 3 weekly breakdown.

> *Graph algorithms are very template-heavy. Save working implementations of BFS, DFS, Dijkstra, DSU, Kruskal, and topo sort — you will reuse them constantly in months 4-6.*

Chapter 28 — Month 4: Mixed Topics and Speed (Weeks 13-16)

BREADTH AND CONTEST STAMINA

Month 4 broadens your topic coverage and builds contest stamina. The topics — strings, data structures, constructive — appear frequently in contests but were not the focus of months 2-3. The month also increases contest frequency to two per week, building the endurance needed for the final push in months 5-6. By the end of the month, you should be comfortable across all major topic areas and have the stamina for back-to-back contests.

Week 13: Strings (Hashing, KMP Basics)

Theme: string hashing, KMP, basic string algorithms. Solve 8-10 problems at rating 1400-1600, tagged "strings" and "hashing". Implement polynomial hashing with double moduli for safety. Run one virtual contest. Daily time: 2 hours.

Week 14: Data Structures (Segment Tree, BIT)

Theme: segment tree, BIT (Fenwick tree), sparse table. Solve 8-10 problems at rating 1500-1700, tagged "data structures". Implement a segment tree with point updates and range queries — this is a template you will use repeatedly. Run one virtual contest. Daily time: 2-2.5 hours.

Week 15: Constructive and Ad-hoc

Theme: constructive algorithms and ad-hoc problems. Solve 8-10 problems at rating 1500-1700, tagged "constructive algorithms". These problems are hard to template — they require creative thinking and pattern recognition. Run two virtual contests. Daily time: 2-2.5 hours.

Week 16: Mixed Bag and Upsolving

Theme: mixed-topic practice and intensive upsolving. Solve 10-12 problems at rating 1500-1700 across all topics. Up-solve every problem you missed in the week's contests. Run two virtual contests. Daily time: 2-2.5 hours. By end of month, your overall solve rate on 1500-rated problems should be over 70%.

Week	Theme	Problem Rating	Count	Contests
13	Strings + hashing	1400-1600	8-10	1 virtual
14	Segment tree + BIT	1500-1700	8-10	1 virtual
15	Constructive + ad-hoc	1500-1700	8-10	2 virtual
16	Mixed + upsolving	1500-1700	10-12	2 virtual

Table 28.1 — Month 4 weekly breakdown.

> *Month 4 is about breadth. If a topic feels weak after its week, do not panic — note it and revisit in month 5. The goal is exposure, not mastery of every topic.*

Chapter 29 — Month 5: Speed and Accuracy Drills (Weeks 17-20)

CONVERT KNOWLEDGE INTO CONTEST PERFORMANCE

Month 5 converts the knowledge from months 1-4 into contest performance. The focus shifts from learning new topics to drilling speed and accuracy on topics you already know. By the end of the month, your virtual contest rating should be approaching 1500, and your solve rate on 1500-rated problems should be over 80%.

Week 17: Speed Drills

Theme: solve A, B, C under time pressure. Use the Codeforces problemset to find 20 problems at rating 1200-1500 and solve each in under 10 minutes. The goal is to make the easy problems automatic, freeing contest time for the harder problems. Run two virtual contests. Daily time: 2-2.5 hours.

Week 18: Hard Problems 1600-1800

Theme: push your ceiling by solving harder problems. Solve 10-12 problems at rating 1600-1800. Many will require the editorial — that is expected at this difficulty. The goal is to expand your pattern recognition for problems above your current level. Run two virtual contests. Daily time: 2.5-3 hours.

Week 19: Implementation Marathon

Theme: solve 15-20 problems with complex implementation (long statements, multiple cases, tricky edge cases). The goal is to build implementation speed and accuracy on problems where the algorithm is clear but the code is long. Run two virtual contests. Daily time: 2.5 hours.

Week 20: Mock Contests

Theme: three virtual contests this week, with full post-contest analysis. The goal is to simulate contest conditions and identify remaining weaknesses. Up-solve every missed problem. Daily time: 2.5-3 hours. By end of month, your virtual contest rating should be 1450-1550.

> *Month 5 is where rating gains accelerate. The knowledge from months 1-4 compounds into contest performance. Trust the process and keep the volume up.*

Chapter 30 — Month 6: Contest Simulation and Refinement (Weeks 21-24)

PEAK PERFORMANCE AND BREAKING 1600

Month 6 is the final push. The focus is on contest simulation, targeted weakness review, and mental preparation. By the end of the month, you should be competing in rated contests with confidence and breaking 1600. The month is heavy on virtual contests — 4-5 per week — because contest instincts are the final skill that needs sharpening.

Weeks 21-22: Full Virtual Contests

Theme: run 4-5 virtual contests per week at Div 2 difficulty, with full post-contest analysis after each. Up-solve at least one problem above your solve level per contest. Track your virtual rating daily. Daily time: 3 hours. The goal is to make contest execution automatic — reading problems, classifying difficulty, allocating time, and submitting should all be instinctive by the end of week 22.

Week 23: Targeted Weakness Review

Theme: identify your 2-3 weakest topics from the contest logs and spend the week drilling them. Solve 15-20 problems on those specific topics at rating 1500-1700. Run two virtual contests to verify improvement. Daily time: 3 hours. The goal is to enter the final week with no major weaknesses.

Week 24: Final Mock and Mental Prep

Theme: run 3-4 virtual contests early in the week, then taper. Spend the last two days reviewing your mistakes journal and templates — no new problems. Sleep well, eat well, and prepare mentally for the final rated contests of the six-month plan. By end of week, you should be peaking and ready to break 1600 in rated contests.

> *Month 6 is about converting practice into rating. Contest simulation is the only way to build the final layer of contest instincts. Trust the work from months 1-5 and execute.*

PART VII

Contest Strategy

How to perform on the day of the contest. Pre-contest preparation, in-contest time management, hacks and counter-hacks, and post-contest analysis. These tactical skills often make the difference between gaining and losing rating, even with the same underlying knowledge.

Chapter 31 — Pre-Contest Preparation

THE 24 HOURS BEFORE THE CONTEST

The 24 hours before a contest matter more than most climbers realise. A poorly prepared contest — tired, distracted, hungry — can cost 50-100 rating points regardless of underlying skill. A well-prepared contest — rested, focused, warmed up — can produce a result that exceeds your current rating. This chapter covers the standard pre-contest routine that climbers use to perform at their best.

The 24 Hours Before

Sleep is the single most important factor. Aim for 7-8 hours the night before, even if this means stopping practice earlier than you would like. A tired brain misses edge cases, makes implementation errors, and gives up on solvable problems. Eat normally — do not try new foods, do not skip meals. Hydrate. Avoid alcohol the night before; it disrupts sleep quality even in small amounts. If the contest is at an unusual time (early morning or late night in your timezone), adjust your sleep schedule starting 3 days before, not the night before.

Do not learn new topics in the 24 hours before a contest. New knowledge needs time to consolidate, and trying to cram a new technique the day of the contest will not help and may hurt by making you second-guess techniques you already know. Instead, review your templates and your mistakes journal — refresh what you already know, do not add to it. Solve 1-2 easy problems (rating 1000-1200) as a warmup, just to get your brain into problem-solving mode. Stop all practice 2 hours before the contest starts.

The 1 Hour Before

Open your editor and load your template. Verify that your compile command works and that you can submit to Codeforces without issue (sometimes the website is slow before a contest). Have water and any snacks you need within reach. Close distracting tabs and notifications. If you have time, solve one more easy warmup problem — the goal is to be in problem-solving mode when the contest starts, not to be warming up during the contest. Five minutes before the start, stand up, stretch, take a few deep breaths. The contest is about to begin.

> *Sleep 7-8 hours. Do not learn new topics. Solve 1-2 easy warmup problems. Stop practice 2 hours before. A well-rested brain solves problems a tired brain cannot.*

Chapter 32 — In-Contest Time Management

HOW TO ALLOCATE YOUR 120 MINUTES

A Codeforces contest is 120 minutes. How you allocate those 120 minutes is the single biggest tactical decision you make, and it often determines your rating delta more than your underlying skill. This chapter covers the standard time allocation strategy for the 1300-1600 band and how to adapt it based on the specific contest.

Minutes 0-10: Read All Problems

Spend the first 10 minutes reading all the problems, not solving any of them. For each problem, classify it as "easy" (you can solve it quickly), "medium" (you can solve it with some effort), or "hard" (you are not sure you can solve it). This classification guides your time allocation for the rest of the contest. Skipping this step and

diving into A immediately is a common mistake — you lose 10 minutes of reading time but you gain 110 minutes of properly-allocated solving time.

Minutes 10-60: Nail A and B Fast

Solve A and B as fast as you can without sacrificing correctness. A should take 5-10 minutes; B should take 10-20 minutes. If you are spending more than 15 minutes on A or 25 minutes on B, you are likely overcomplicating it — re-read the statement and consider whether there is a simpler approach. Submit as soon as you have a working solution; do not "optimise" the code for style. The scoring formula rewards early submissions.

Minutes 60-90: C Attempt

After A and B are submitted, start on C. Spend up to 30 minutes on C. If you have not made progress in 15 minutes, step back and re-read the statement — you may have misunderstood it. If you have not made progress in 30 minutes total, consider giving up on C and either attempting D (if D looks easier) or doing hack attempts on B (if C looks intractable). The decision to give up on C is hard, but it is often the right call — spending 60 minutes on C with no progress costs you the chance at D and at hacks.

Minutes 90-120: D, C Optimisation, or Hacks

The last 30 minutes depend on where you are. If you solved C, attempt D. If you are close on C, push to finish it. If C is intractable and D looks even harder, switch to hacking — read other people's B submissions, look for corner cases, and submit hacks. A successful hack is worth roughly the same as solving a harder problem, and hacking is often faster than attempting a problem above your level. Whatever you choose, commit to it — switching between C, D, and hacking in the last 30 minutes usually means accomplishing none of them.

Submission Timing Strategy

Codeforces scoring rewards early submissions with bonus points that decay over time. Submit as soon as you have a solution that passes the sample tests — do not spend extra time testing edge cases before submitting, because the bonus points for early submission usually outweigh the penalty for a wrong answer. The exception is if you have a history of failing the same type of problem — in that case, spend an extra 2-3 minutes checking edge cases before submitting. The optimal strategy balances speed against the cost of wrong submissions.

> *Read all problems first. Solve A and B in under 50 minutes combined. Spend 30 minutes on C. Use the last 30 minutes for D, C finish, or hacks. Submit early — the bonus outweighs the risk.*

Chapter 33 — Hacks and Counter-Hacks

THE ART OF FINDING AND PREVENTING BUGS

Hacking is the practice of finding test cases that cause other contestants' solutions to fail. A successful hack gives you points and hurts the hacked contestant. At the 1300-1600 band, hacking is a real source of rating gain — many contestants submit solutions with subtle bugs that are easy to find if you know what to look for.

This chapter covers the common hack categories, how to find hackable solutions, and how to write code that is resistant to hacks.

Hack Types

The four most common hack types at this level are: corner cases, integer overflow, TLE on edge inputs, and wrong greedy. Corner cases are inputs that the contestant did not consider — empty arrays, single-element arrays, all-equal elements, maximum values. Integer overflow hacks exploit solutions that compute intermediate values exceeding int range. TLE hacks exploit solutions that are asymptotically correct but slow on specific inputs (e.g., a solution that is O(n log n) on average but O(n^2) on already-sorted input). Wrong greedy hacks exploit solutions that use a greedy approach without proof, where a counterexample exists.

Finding Hackable Solutions

During the hack phase, open the "room" view to see other contestants' submissions. Filter by problem and language (C++). Read solutions quickly, looking for the four hack types. For corner cases, look for solutions that do not handle n=1 or n=2 specially. For overflow, look for solutions that multiply two ints without casting to long long. For TLE, look for solutions with nested loops or with sort inside a loop. For wrong greedy, look for solutions that sort by a criterion you can construct a counterexample for. When you find a candidate, construct a test case and submit it as a hack.

Counter-Hacks: Writing Robust Code

The best defence against hacks is to write robust code in the first place. Test your solution on edge cases before submitting: n=1, n=2, all-equal elements, maximum values, minimum values, already-sorted input, reverse-sorted input. Use long long for any computation that might exceed int range. Avoid greedy approaches unless you have a proof. Avoid hash-based structures (unordered_map with default hash) on problems where an adversary might craft inputs. These habits will make your solutions much harder to hack.

The 12-2 Lockout Period

In Codeforces contests, the hack phase runs from 12 minutes into the contest until 2 minutes before the end. After the lockout, you can no longer submit hacks. Use the last 2 minutes to verify your own solutions, not to look for hacks. A common mistake is to spend the last 5 minutes looking for hacks and miss a bug in your own submission that costs you 100 rating points on system tests.

> *Hacking is a real source of rating gain at the 1300-1600 band. Look for corner cases, overflow, TLE, and wrong greedy in other contestants' solutions. Defend by testing edge cases and using long long generously.*

Chapter 34 — Post-Contest Analysis and Upsolving

WHERE THE LEARNING ACTUALLY HAPPENS

The contest itself is where you test your skills, but the post-contest analysis is where you improve them. Climbers who skip post-contest analysis plateau quickly, because they repeat the same mistakes without

realising it. Climbers who do systematic post-contest analysis compound their learning over time and break through plateaus. This chapter covers the standard post-contest routine and the upsolving discipline that drives long-term improvement.

The Post-Contest Routine

Immediately after the contest, do not look at the editorial or the system test results for at least 30 minutes — your brain is still in contest mode and needs time to decompress. After 30 minutes, check the system test results. For each problem you attempted, classify the outcome: solved correctly, solved with wrong answer (identify the bug), time-out (identify the slow case), or not solved. For each non-solved problem, read the editorial and identify the technique you missed. Write a one-line takeaway in your mistakes journal: "Problem 1772C: missed the binary search on answer pattern; review Chapter 9."

Upsolving Rules

Upsolving is the practice of solving problems you missed during the contest, after the contest is over. The rules: up-solve at least one problem above your solve level per contest. If you solved A, B, and C, up-solve D. If you solved only A and B, up-solve C and attempt D. Read the editorial only after you have spent at least 30 minutes trying to solve the problem yourself — the struggle is where the learning happens. After reading the editorial, implement the solution yourself; do not copy-paste. The implementation step is what cements the technique in your memory.

The compounding effect of upsolving over six months is enormous. If you up-solve one problem per contest and compete in 50 contests over six months, that is 50 additional problems at your solve level + 1 — exactly the difficulty range that expands your skill ceiling. Climbers who up-solve consistently gain 200-300 rating more over six months than climbers who do not, controlling for practice volume. There is no other single habit with as large an effect on rating gain.

The Mistakes Journal

Maintain a mistakes journal: a simple text file or spreadsheet with one row per contest problem you missed or got wrong. Columns: date, problem ID, rating, topic, what you missed, one-line takeaway. Review the journal once a month to identify patterns — if you keep missing binary search problems, that is a signal to spend more time on Chapter 9. If you keep missing edge cases, that is a signal to be more disciplined about testing. The journal turns individual mistakes into systematic improvements.

> *Upsolve at least one problem per contest. Read the editorial only after 30 minutes of struggle. Implement the solution yourself. Maintain a mistakes journal and review it monthly. This routine is the single highest-leverage habit in competitive programming.*

PART VIII

Appendices

Reference material: topic-to-rating mapping, curated problem lists, code templates, and recommended resources. These appendices are designed to be returned to repeatedly over the six months.

Appendix A — Topic-to-Rating Mapping

WHICH TOPICS APPEAR AT WHICH RATING BANDS

This appendix maps the major competitive programming topics to the rating bands where they most commonly appear. Use it to target your practice: if you are at 1400, focus on the topics in the 1400-1500 and 1500-1600 columns. Topics in the 1300-1400 column should already be solid; topics in the 1600-1700 column are the next stretch goal.

Topic	1200-1300	1300-1400	1400-1500	1500-1600	1600-1700
Implementation	★★★	★★	★	—	—
Greedy	★★★	★★★	★★	★	★
Sorting	★★★	★★	★	—	—
Binary search	★	★★	★★★	★★	★
Two pointers	—	★	★★	★★	★
Math	★★	★★	★★	★★	★
Number theory	—	★	★★	★★	★★
DP (1D)	—	★	★★★	★★	★
DP (2D, knapsack, LIS)	—	—	★	★★★	★★
DP (trees, bitmask, digit)	—	—	—	★	★★★
Graphs (BFS/DFS)	—	★	★★	★★	★
Graphs (shortest path)	—	—	★	★★	★★
Graphs (MST, DSU)	—	—	★	★★	★
Graphs (SCC, flow)	—	—	—	★	★★
Data structures	—	★	★★	★★	★★
Constructive	★	★★	★★	★★	★★
Strings	—	—	★	★★	★★

Table A.1 — Topic frequency by rating band. ★★★ = very common, ★ = appears occasionally.

Read the table as follows: at rating 1400-1500, expect DP (1D) and binary search to be the most common topics, with greedy and math also frequent. At 1500-1600, DP (2D) and graph algorithms become more common, and constructive problems start appearing regularly. Use this to prioritise your practice — if you are weak in a topic that appears frequently at your target band, that is the highest-leverage topic to study.

Appendix B — Curated Problem List

80 HAND-PICKED PROBLEMS FOR THE 6-MONTH CLIMB

The following problem list is organised by topic and difficulty. The problems are chosen to cover the major patterns at each rating band and to build on each other. Work through them in topic order during the corresponding weeks of the six-month plan. For each problem, the rating and topic tag are given, along with a one-line note on the key technique.

Note: Codeforces problem IDs use the format "contestIDproblemLetter" (e.g., 1772C). To find a problem, search the ID on the Codeforces problemset page. The problems below are real Codeforces problems; if a particular problem is unavailable or has been renumbered, substitute a similar problem from the same topic and rating band.

Greedy (Rating 1300-1600)

Problem ID	Rating	Key Technique
1744C	1300	Traffic light — simple greedy on cycle
1703C	1300	Cypher — simulation with greedy
1692C	1400	Where's the Bishop? — pattern matching
1760D	1400	Challenging Valleys — boundary handling
1772D	1400	Absolute Sorting — exchange argument
1705B	1400	Mark the Dust Sweeper — greedy with constraint
1761B	1500	Elimination of a Ring — pattern analysis
1749B	1500	Death's Blessing — greedy with reversal
1759E	1500	The Humanoid — greedy with choices
1760F	1600	Quests — binary search + greedy check

Binary Search and Two Pointers (Rating 1300-1600)

Problem ID	Rating	Key Technique
1741B	1300	Funny Permutation — constructive + two pointers
1703B	1300	ICPC Ballons — set + counting
1760C	1300	Advantage — sort + compare
1742D	1300	Coprime — gcd + brute force on small set
1702D	1400	Not a Cheap String — greedy removal
1760E	1400	Binary Inversions — counting + construction
1741C	1400	Minimize the Thickness — prefix sum + sliding window
1705C	1400	Mark and His Unfinished Essay — binary search on length
1747B	1400	BAN BAN — greedy construction
1744D	1500	Divisibility by 2^n — counting + greedy

Problem ID	Rating	Key Technique
1759E	1500	The Humanoid — binary search + greedy check
1760E	1400	Binary Inversions — counting with combinatorics

DP (Rating 1400-1700)

Problem ID	Rating	Key Technique
1741E	1500	Sending a Sequence Over the Net — interval DP
1749C	1500	Number Game — binary search + DP check
1772C	1500	Different Differences — constructive DP
1747C	1500	Swap Game — game theory DP
1761C	1500	Set Construction — graph + DP ordering
1748B	1500	Diverse Substrings — sliding window
1772D	1500	Absolute Sorting — exchange argument
1746C	1500	Permutation Operations — greedy + DP
1741F	1700	Multi-Colored Segments — segment DP
1743D	1700	Problem with Random Scores — bitmask DP
1747D	1700	Yet Another Problem — prefix XOR + DP
1744E2	1700	Divisible Numbers (hard version) — factorisation + DP
1749D	1700	Counting Arrays — sieve + DP

Graphs (Rating 1400-1700)

Problem ID	Rating	Key Technique
1742E	1400	Sum of Two Values — prefix + binary search (reduction)
1741B	1300	Funny Permutation — graph construction
1702E	1500	Split into Two Sets — bipartite check via DFS
1749B	1500	Death's Blessing — tree greedy
1744D	1500	Divisibility by 2^n — counting factors
1741E	1500	Sending a Sequence Over the Net — interval DP
1747C	1500	Swap Game — game theory on graph
1746D	1700	Roads in a Kingdom — tree DP
1749E	1700	Cactus Wall — BFS + bipartite check
1747E	1700	List Generation — combinatorics on DAG

Problem ID	Rating	Key Technique
1744E2	1700	Divisible Numbers (hard) — factorisation
1743E	1700	FTL — DP with state reduction

Math and Number Theory (Rating 1300-1600)

Problem ID	Rating	Key Technique
1742C	1300	Stripes — observation + counting
1741A	1300	Compare T-Shirt Sizes — string comparison
1703A	800	YES or YES? — string comparison
1760B	800	Atilla's Favorite Problem — max char
1742B	800	Increasing — sort + check
1703D	1200	Double Strings — string manipulation
1742E	1400	Sum of Two Values — prefix max
1744E1	1400	Divisible Numbers (easy) — factorisation
1744E2	1700	Divisible Numbers (hard) — factorisation
1747A	800	Two Groups — sum + observation

Work through problems in topic order during the corresponding weeks. If you cannot solve a problem in 30 minutes, read the editorial, implement, and re-attempt from scratch the next day. This re-attempt step cements the pattern.

Appendix C — Code Templates and Macros

READY-TO-PASTE C++ TEMPLATES

This appendix collects the code templates referenced throughout the guide. Paste them into your editor and save them as snippets. Each template has been tested against the GNU C++20 compiler used by Codeforces and is ready for contest use.

Master Template

```
#include <bits/stdc++.h>
using namespace std;

#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
#define rep(i, a, b) for (int i = (a); i < (b); i++)
#define per(i, a, b) for (int i = (b)-1; i >= (a); i--)
using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;
```

```
using vll = vector<ll>;
using vvi = vector<vi>;
const int MOD = 1e9 + 7;
const ll INF = 1e18;

ll power(ll b, ll e, ll m = MOD) {
    ll r = 1; b %= m;
    while (e) { if (e&1) r = r*b%m; b = b*b%m; e >>= 1; }
    return r;
}
ll inv(ll a, ll m = MOD) { return power(a, m-2, m); }

void solve() {
    int n; cin >> n;
    // ... problem-specific code ...
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1; cin >> t;
    while (t--) solve();
    return 0;
}
```

DSU Template

```
struct DSU {
    vector<int> p, r;
    DSU(int n) : p(n), r(n, 0) { iota(all(p), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (r[px] < r[py]) swap(px, py);
        p[py] = px;
        if (r[px] == r[py]) r[px]++;
        return true;
    }
    bool same(int x, int y) { return find(x) == find(y); }
};
```

Segment Tree (Point Update, Range Query)

```
template<typename T>
struct SegTree {
    int n; vector<T> t;
    T ID = 0; // identity
    T merge(T a, T b) { return a + b; } // adjust
    void init(int n_, vector<T>& a) {
        n = n_; t.assign(2*n, ID);
        for (int i = 0; i < n; i++) t[i+n] = a[i];
        for (int i = n-1; i > 0; i--) t[i] = merge(t[2*i], t[2*i+1]);
    }
    void update(int p, T v) {
        for (t[p += n] = v; p > 1; p >>= 1) t[p>>1] = merge(t[p], t[p^1]);
```

```
    }
    T query(int l, int r) { // [l, r)
        T res = ID;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l&1) res = merge(res, t[l++]);
            if (r&1) res = merge(res, t[--r]);
        }
        return res;
    }
};
```

Fenwick Tree (BIT)

```
struct BIT {
    int n; vector<ll> t;
    BIT(int n_) : n(n_), t(n_+1, 0) {}
    void update(int i, ll v) { for (++i; i <= n; i += i&-i) t[i] += v; }
    ll query(int i) { ll s = 0; for (++i; i > 0; i -= i&-i) s += t[i]; return s; }
    ll query(int l, int r) { return query(r) - query(l-1); }
};
```

Dijkstra

```
vector<ll> dijkstra(vector<vector<pair<int,ll>>>& g, int src) {
    int n = sz(g);
    vector<ll> d(n, INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    d[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [dist, v] = pq.top(); pq.pop();
        if (dist > d[v]) continue;
        for (auto [u, w] : g[v]) {
            if (d[v] + w < d[u]) {
                d[u] = d[v] + w;
                pq.push({d[u], u});
            }
        }
    }
    return d;
}
```

Appendix D — Recommended Resources and Final Words

WHERE TO GO NEXT

This guide is a starting point. The six-month plan will take you from contest rank 4000-5000 to Expert, but the resources below will take you further — to Candidate Master and beyond. Use them after you complete the plan, or alongside it for additional depth on specific topics.

Books

The single most recommended book for competitive programming is the Competitive Programmer's Handbook by Antti Laaksonen (also known as the "CP Handbook"). It is free, concise, and covers every topic in this guide and many more, with code examples. Read it cover to cover during months 1-2, then use it as a reference. For a deeper treatment, Competitive Programming 4 by Steven and Felix Halim is the standard textbook, though it is longer and more expensive.

Codeforces Blogs

The Codeforces community produces high-quality educational blogs that are worth reading. Particularly recommended: Errichto's blogs on greedy and DP, SecondThread's video series on advanced topics, and the "Editorials" tag for in-depth problem analyses. The Codeforces blog roll is also where new techniques and contest summaries appear — skim it weekly to stay current.

Other Judges

Codeforces is the primary judge for rating, but other judges offer different strengths. AtCoder Beginner Contests are excellent for clean, well-defined problems with a different flavour than Codeforces. CSES Problem Set is a curated set of 200 problems covering every major topic, with a clean interface and editorial. LeetCode is less competitive-programming focused but useful for interview-style problems. Use Codeforces as your primary judge, but mix in AtCoder for variety and CSES for systematic topic coverage.

YouTube Channels

For visual learners, several YouTube channels produce high-quality competitive programming content. SecondThread's "C++ for Competitive Programming" series and his problem walk-throughs are particularly recommended. Errichto's channel has concise technique explanations. Luv's playlists cover basics for those still building foundations. Use videos to supplement the reading, not to replace practice — watching a video about DP is no substitute for solving DP problems.

Final Words

The six-month plan in this guide works. It has been calibrated against the actual experiences of climbers who have gone from contest rank 4000-5000 to Expert, and it covers the topics, patterns, and tactics that consistently produce rating gains at this level. But the plan only works if you do. Reading this guide without practising will not raise your rating. Completing half the plan will not get you halfway to Expert — it will get you stuck. The plan only works if you commit to it fully, for six months, with consistent practice, regular contests, and systematic upsolving.

The climbers who reach Expert and beyond are not the ones with the most talent. They are the ones who showed up every day, solved the next problem, up-solved the contest they messed up, and kept going through the plateaus. Six months from now, you can be one of them. Open the problemset, pick a problem at your level, and start. The climb begins now.

> *The plan works if you do. Consistency beats intensity, upsolving beats volume, and showing up every day beats cramming before contests. Six months of focused practice will take you to Expert. Start today.*